Artificial Intelligence-Enabled Digital Twin for Intelligent Building Monitoring and Predictive Analytics

A PROJECT REPORT

Submitted by

DHANUSH A 111923AI01023

in partial fulfillment for the award of the degree

of

B. TECH

IN

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

S. A. ENGINEERING COLLEGE, CHENNAI - 600077

(Autonomous - Institute Level Research Centre, Affiliated to Anna University)

ANNA UNIVERSITY : CHENNAI 600 025

NOVEMBER 2026

BONAFIDE CERTIFICATE

Certified that this project report **Artificial Intelligence-Enabled Digital Twin for Intelligent Building Monitoring and Predictive Analytics** is the bonafide work of **DHANUSH A (111923AI01023)**
who carried out the project work under my supervision.

SIGNATURE OF HOD

HEAD OF THE DEPARTMENT
Dr. D. HEMANAND, M.E., PhD.,
Professor & Head,
DEPARTMENT OF AI & DS,
S.A.ENGINEERING COLLEGE,
Avadi- Poonamallee Main Road,
Thiruverkadu Post,
Chennai-600 077

SIGNATURE OF SUPERVISOR

SUPERVISOR
Mr. G. MUTHU KUMAR, M.E.,(PhD).,
Assistant Professor ,
DEPARTMENT OF AI & DS,
S.A.ENGINEERING COLLEGE,
Avadi-Poonamallee Main Road,
Thiruverkadu Post,
Chennai-600 077

Submitted to Project Viva-Voice Examination held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We owe a great many thanks to a great many people who helped and support us during the completion of our project.

We take this opportunity to express our profound gratitude and deep regards to our Founder Chairman **(Late)** Shri**. D. SUDHARSSANAM, Ex-M.L.A,** our Chairman Shri**. D. DURAISWAMY**, our humble Secretary Shri**. D. DASARATHAN** our correspondent Shri. **S. AMARNAATH**, and our Director Shri**. D. SABARINATH**, for their exemplary guidance, monitoring and constant encouragement throughout the course of this thesis. The blessing, help and guidance given by them time to time shall carry us a long way in the journey of life on which we are about to embark.

We also take this opportunity to sincerely thank our beloved Principal **Dr. S. RAMACHANDRAN, M.E., Ph.D.,** for extending his support and guidance. Also we express a deep sense of gratitude to **Dr. D. HEMANAND, M.E., Ph.D.,** Head of the Department of Artificial Intelligence and Data Science for his guidance and encouragement in our project work which helped us in completing this task through various stages.

We are also obliged to convey our special and sincere gratitude to our project coordinators, **Mrs. B. ARDLY MELBA REENA, M.E., (Ph.D).,** Assistant Professor, AI & DS Department, **Dr K SUBHA, M.E., Ph.D.,** Assistant Professor, AI & DS Department, and our internal guide, **Mr. G. MUTHU KUMAR, M.E., (PhD),** AI & DS Department, for their encouragement and concern during the review session, along with their valuable advice, knowledge, and expert guidance in our project.

Lastly, we thank almighty, our parents, all the teaching and non-teaching staff for their support and guidance throughout our project and for their constant encouragement without which this project would have not been possible.

ABSTRACT

Constant monitoring of environmental conditions and occupant activities in various spaces is essential for effective building management. Traditional monitoring systems can collect sensor measurements but have limitations in organizing the data for analysis and visualization in a timely manner. This paper presents an AI-enabled Digital Twin system that integrates the Internet of Things (IoT), machine learning, and rule-based analysis with real-time data processing and Digital Twin visualization for intelligent building monitoring. The system is built on an Espressif Systems ESP32 microcontroller and is equipped with DHT22 and MQ-2 sensors, as well as PIR motion sensors, to capture environmental conditions in three spaces of the prototype building: Reception, Manager Cabin, and Workspace. The measured data are transmitted to a FastAPI backend via Wi-Fi for storage in a MongoDB database. For analysis, the data are preprocessed, and a machine learning model based on Random Forest is developed to predict room occupancy, while rule-based analysis classifies environmental conditions into normal, warning, and critical states. PIR sensors also provide occupancy-related information. A Digital Twin dashboard is developed using React and provides occupancy-related information along with sensor measurements, historical data, alerts, and condition status of the monitored spaces. The integrated prototype establishes a basis for building automation and monitoring applications and demonstrates the integration of various technologies, including IoT, AI/ML, and Digital Twin.

***Keywords*—**Digital Twin, Internet of Things, Smart Building, Machine Learning, Random Forest, ESP32, Real-Time Monitoring, Occupancy Prediction, Anomaly Detection, Building Monitoring.

TABLE OF CONTENTS

Chapter No	Title	Page No
	ABSTRACT	IV
	LIST OF TABLE	VIII
	LIST OF FIGURES	IX
	LIST OF ABBREVIATIONS	X
1.	**INTRODUCTION**	**1**
	1.1 Overview	1
	1.2Importance of the project	1
	1.3Objectives	2
	1.4Scope of the project	3
	1.5Applications	3
2.	**LITERATURE SURVEY**	4
3.	**PROPOSED SYSTEM**	7
	3.1 Existing System	7
	3.2 Disadvantages	7
	3.3 System Ojectives	8
	3.4 Proposed Features & Functionalities	9
	3.4.1 IoT-Based Data Acquisition Module	9
	3.4.2 Data Preprocessing Module	9
	3.4.3 AI/ML Prediction Module	9
	3.4.4 Anomaly Detection Module	9
	3.4.5 Digital Twin Visualization Module	9
	3.4.6 Real-Time Monitoring and Alert	9

Module

3.4.7 Analytics and Reporting Module 9

3.4.8 System Integration and Scalability Module 10

3.5 System Architecture 10

3.6 Advantages of the Proposed System 11

4. **REQUIREMENT SPECIFICATION** 13

4.1 Hardware Requirements 13

4.2 Software Requirements 13

4.3 Development Environment 14

4.4 Technology Stack 14

4.5 Functional Requirements 15

4.6 Non-Functional Requirements 15

5. **SYSTEM DESIGN** 17

5.1 Proposed Model 17

5.2 System Architecture Design 17

5.3 IoT Sensor and Data Collection Design 18

5.4 Data Preprocessing 19

5.5 AI/ML Model Design 19

5.6 Anomaly Detection Design 20

5.7 Digital Twin Design 21

5.8 Database Design 21

5.9 Dashboard Design 22

5.10 Deployment Design 23

6. **IMPLEMENTATION** 25

6.1 Overview 25

6.2 Hardware Implementation 26

6.3 IoT Data Acquisition and Communication 27

6.4 Backend Implementation 28

6.5 Database Implementation 26

6.6 Data Preprocessing Implementation 30

6.7 AI/ML Implementation 30

6.8 Anomaly Detection Implementation 31

6.9 Digital Twin Implementation 32

6.10 Frontend and Dashboard Implementation 33

6.11 System Integration 34

6.12 Implementation Summary 34

7. **RESULTS AND DISCUSSION** 36

7.1 Overview 36

7.2 Model Performance 36

7.3 Real-Time Recommendations 37

7.4 Discussion 38

7.5 Data Visualizing and Reporting 41

8. **CONCLUSION AND FUTUREWORK** 43

8.1 Conclusion 43

8.2 Framework for Future Work 44

APPENDIX 1 (SOURCE CODE) 45

APPENDIX 2 (SNAP SHOT) 61

REFERENCES 63

LIST OF TABLES

Table.No	Title	Page no
7.4.1	Implemented Machine Learning Models	40
7.4.2	Model Input Parameters and Outputs	40
7.4.3	AI/ML Results Available from the System	41

LIST OF FIGURES

Figure No	Title	Page No
3.5.1	Architecture Model	11
5.8.1	End-to-End Workflow of the Platform	22
6.2.1	Hardware setup	26
6.3.1	Hardware Communication	27
6.4.1	Backend Connectivity	29
A2.1	Hardware Prototype	61
A2.2	Digital Twin dashboard for real-time building monitoring and visualization	61
A2.3	Room-level Digital Twin dashboard showing environmental and Occupancy condition	61
A2.4	Historical sensor trends and analytical information displayed in the Digital Twin dashboard.	62
A2.5	Processed sensor conditions and alert states for the monitored rooms	62

LIST OF ABBREVIATIONS

ABBREVIATION	FULL FORM
AI	Artificial Intelligence
BIM	Building Information Modeling
DT	Digital Twin
IoT	Internet of Things
ML	Machine Learning
ESP32	Espressif 32-bit Microcontroller
API	Application Programming Interface
HVAC	Heating, Ventilation, and Air Conditioning
UI	User Interface
RF	Random Forest
LSTM	Long Short-Term Memory
XGBoost	Extreme Gradient Boosting
MAE	Mean Absolute Error
RMSE	Root Mean Square Error
R^2	Coefficient of Determination

CHAPTER 1
INTRODUCTION

1.1 Overview

The rapid growth of smart building technologies has increased the adoption of Internet of Things (IoT) devices for monitoring environmental conditions, occupancy, and operational activities. However, conventional building monitoring systems often provide only real-time sensor values and require manual analysis to identify abnormal conditions or potential issues. Digital Twin technology provides an effective solution by creating a virtual representation of physical building spaces and continuously reflecting their real-world conditions through connected sensors. This project focuses on developing an AI-enabled Digital Twin system for intelligent building monitoring and predictive analytics using IoT technologies. The proposed system integrates ESP32-based sensors to collect temperature, humidity, gas, and motion data from different building spaces and transmits the information to the backend for storage and analysis. Artificial Intelligence and Machine Learning techniques are applied to analyze sensor data, identify unusual patterns, and generate predictive insights that support intelligent decision-making. Unlike conventional monitoring approaches, the proposed system combines real-time sensing, data processing, AI-based analytics, and Digital Twin visualization within a unified framework. By continuously analyzing collected data, the system can detect anomalies, understand changing environmental conditions, and provide timely information for building management. The Digital Twin dashboard presents the monitored conditions in an intuitive manner, enabling users to observe individual spaces and understand their current status. Ultimately, the system aims to provide scalable, real-time, and intelligent building monitoring that improves situational awareness, supports predictive analysis, and enhances efficient management of building environments.

1.2 Importance of the project

The importance of this project lies in improving building monitoring through the integration of Internet of Things (IoT), Artificial Intelligence, and Digital Twin technologies. Modern buildings contain multiple spaces where environmental conditions can change continuously due to occupancy, equipment operation, and external factors. Effective monitoring of these conditions is essential not only for maintaining a comfortable and safe environment but also for supporting efficient building management and timely decision-making. Conventional monitoring systems generally display

sensor readings but provide limited intelligence for identifying abnormal conditions or predicting future changes. To overcome these limitations, this project integrates AI and Machine Learning techniques with a Digital Twin framework to analyze continuously collected building data and identify meaningful patterns. The system uses ESP32-based sensors to collect temperature, humidity, gas, and motion information from different building spaces and transfers the data for processing and analysis. AI-based analysis helps identify unusual variations and generate predictive insights that can assist users in understanding changing building conditions. The Digital Twin provides a visual representation of the monitored spaces, allowing real-time conditions to be observed in an intuitive manner. By combining sensing, data processing, intelligent analysis, and visualization, the proposed system provides a more proactive approach to building monitoring compared with conventional systems. This integrated architecture can improve situational awareness, support early identification of abnormal conditions, and provide useful information for effective building management. Ultimately, the project contributes toward intelligent, scalable, and data-driven building environments.

1.3 Objectives

The primary objectives of this project are: To design and develop an AI-enabled Digital Twin system for intelligent monitoring of building environments using IoT technologies. To collect real-time temperature, humidity, gas, and motion data from different building spaces using ESP32-based sensor nodes. To preprocess and organize the collected sensor data by handling inconsistencies and preparing suitable inputs for analysis. To apply Artificial Intelligence and Machine Learning techniques for identifying patterns, detecting anomalies, and generating predictive insights from building data. To integrate the processed sensor information with a Digital Twin representation for real-time visualization of individual building spaces. To develop a dashboard that enables users to monitor environmental conditions, observe space status, and understand detected abnormalities. To evaluate the performance of the implemented AI models using suitable performance metrics and observed prediction results. To provide timely alerts and recommendations based on detected abnormal conditions and predicted changes. To develop a scalable system architecture that supports continuous data collection, analysis, visualization, and future integration of additional building monitoring capabilities.

1.4 Scope of the project

The scope of this project is centered on developing an intelligent Digital Twin system for building monitoring and predictive analytics using Internet of Things (IoT) and Artificial Intelligence technologies. The system leverages IoT sensors to collect environmental and occupancy-related information by monitoring temperature, humidity, gas levels, and motion conditions from different building spaces in real time. Its primary scope includes collecting, preprocessing, storing, and analyzing sensor data, applying AI and Machine Learning techniques for identifying patterns and detecting abnormal conditions, and generating useful predictive insights for building management. The project also covers the development of a Digital Twin representation that provides a virtual view of the monitored physical spaces and enables users to observe their current conditions through an interactive dashboard. Furthermore, the scope extends to implementing real-time monitoring, anomaly identification, alerts, and data visualization features to support timely decision-making. To provide meaningful system evaluation, the project considers sensor observations, model performance, anomaly detection results, and visualization outcomes. While the current focus is on monitoring selected building spaces, the system can be scaled and adapted for additional rooms, sensors, analytical models, and larger building environments. Thus, the project provides a comprehensive framework for developing a reliable, scalable, and intelligent building monitoring solution that combines physical sensing, digital representation, and AI-based analytics.

1.5 Application

- Smart Building Monitoring: Real-time monitoring of temperature, humidity, gas, and motion using IoT sensors.
- Environmental Management: Detection of abnormal environmental conditions for timely action.
- Occupancy Monitoring: Monitoring motion activity to understand occupancy and space utilization.
- Predictive Management: AI/ML analysis of sensor data for anomaly detection and predictive insights.
- Digital Twin Visualization: Interactive visualization of building spaces and real-time sensor conditions.
- Facility Management: Centralized monitoring, alerts, and analytical information for administrators.
- Scalable Deployment: Supports additional rooms, sensors, and future intelligent building features.

CHAPTER 2

LITERATURE SURVEY

The integration of Artificial Intelligence (AI), Internet of Things (IoT), and Digital Twin technologies has gained increasing attention in the development of intelligent building monitoring and management systems. Traditional building monitoring systems primarily depend on sensors to collect environmental information and dashboards to display real-time conditions. Although such systems provide useful monitoring capabilities, they generally offer limited predictive analysis and intelligent decision support. Recent research has therefore explored the integration of AI and Machine Learning with Digital Twin environments to support real-time monitoring, predictive analysis, energy optimization, and improved building performance. The following studies discuss relevant developments in Digital Twin architectures, sustainable building environments, AI-based building management, and environmental monitoring.

Alanezi and Mishra [1] proposed a scalable architecture for Building Digital Twins at the edge. Their research focused on supporting Digital Twin applications through edge computing and addressing the processing and communication requirements of building-related data. The study provides an architectural foundation for deploying scalable Digital Twin systems. However, the integration of AI-based anomaly detection and predictive analytics requires additional capabilities beyond the architecture described in the reference title.

Alnaser et al. [2] explored AI-powered Digital Twins and Internet of Things technologies for smart cities and sustainable building environments. Their work connects AI, IoT, and Digital Twin concepts to support intelligent environmental management and sustainability. This research highlights the potential of combining connected sensing systems with digital representations of physical environments. However, the reference title alone does not establish a complete implementation for real-time, multi-parameter monitoring and prediction at the individual-room level.

Anwar et al. [3] proposed an AI-enabled Digital Twin framework for climate-responsive and sustainable building design. Their work focuses on integrating AI and Digital Twin technologies to support sustainable building development and improve building performance. The study demonstrates the relevance of AI-enabled Digital Twins in building-related analysis and decision-making. However,

its stated focus on building design differs from continuous real-time monitoring and predictive management of occupied building spaces.

Boukaf et al. [4] presented a comprehensive review of Digital Twin technology for building energy consumption forecasting. Their research examines the role of Digital Twins in forecasting energy consumption and supporting energy-related management decisions. The study provides useful background on predictive analytics for building performance. However, energy forecasting alone does not cover all the requirements of an integrated environmental monitoring system that also addresses anomaly detection and other building conditions.

Buonomano et al. [5] proposed a Digital Twin framework for optimizing building performance, with particular attention to thermal comfort. Their work highlights the potential of Digital Twin technology to analyze building conditions and support occupant comfort. This approach is relevant to intelligent building management because indoor conditions influence the overall building environment. However, the stated focus on thermal comfort does not establish comprehensive monitoring of additional parameters such as gas concentration and occupancy.

Fang [6] examined the application of Artificial Intelligence and Big Data in smart buildings. The study addresses the relevance of AI and data-driven techniques to smart building applications. Such approaches can provide a foundation for analyzing building-related information and supporting intelligent management decisions. However, the reference details provided do not establish a specific integrated Digital Twin implementation with real-time IoT sensing, predictive analytics, and interactive visualization.

Furuta et al. [7] developed a web-based BIM platform for Building Digital Twin applications. Their work focuses on representing and visualizing building information through a web-based platform. This approach highlights the importance of accessible web technologies for presenting building data and digital representations. However, the reference title does not establish the inclusion of AI-based prediction, anomaly detection, or automated decision support.

Gao et al. [8] presented a Digital Twin-based smart building management system for real-time environmental monitoring and thermal comfort prediction. Their work combines the concepts of digital building representation, environmental monitoring, and predictive analysis. This research is relevant to

systems that aim to move beyond displaying current sensor values toward predicting building conditions. However, the stated focus on environmental monitoring and thermal comfort prediction does not establish coverage of every environmental and operational parameter required by the proposed system.

Ghaemi et al. [9] reviewed AI and Digital Twin applications in building energy management. Their study examined how these technologies can contribute to energy efficiency and intelligent building management. The research highlights the potential of integrating AI-based analytics with Digital Twin environments to support data-driven decision-making and energy optimization. However, the study is a review of existing research rather than a direct implementation of the complete monitoring system proposed in this project.

Hidayat et al. [10] presented a Digital Twin system for smart buildings integrated with blockchain and mixed reality technology. Their work explores the combination of Digital Twin technology with additional digital technologies for smart building applications. The study demonstrates the broader scope of technologies that can be integrated into building-related Digital Twin environments. However, the reference title does not establish a comprehensive implementation of AI-based forecasting and anomaly detection for real-time environmental monitoring.

Overall, these studies demonstrate the relevance of Digital Twin technology, AI, IoT, BIM, web-based visualization, and predictive analytics to intelligent building management [1]–[10]. The reviewed research covers scalable architectures, sustainability, energy forecasting, thermal comfort, smart building applications, and digital visualization. These areas provide a foundation for developing more integrated building monitoring solutions. The proposed project focuses on combining ESP32-based IoT sensing, real-time data collection, backend and database services, AI-based analytics, and an interactive Digital Twin dashboard. The system aims to provide real-time environmental monitoring, anomaly detection, predictive insights, and intelligent decision support for building management.

CHAPTER 3
PROPOSED SYSTEM

3.1 Existing system

In the current scenario, most building monitoring systems used in smart buildings rely on conventional Building Management Systems (BMS) and basic IoT-based monitoring approaches. These systems typically collect parameters such as temperature, humidity, gas levels, and occupancy using sensors and display the information through monitoring interfaces or dashboards.While these methods can provide basic real-time monitoring, they suffer from several limitations. Conventional systems mainly focus on displaying current sensor values and often depend on predefined threshold values or manual observation to identify abnormal conditions. Similarly, traditional monitoring platforms have limited capabilities for analyzing historical sensor data, making it difficult to predict future environmental changes or equipment-related issues. Existing systems also lack intelligent adaptability, limiting their effectiveness in providing proactive building management.Furthermore, most current platforms provide limited integration between IoT data, artificial intelligence, and Digital Twin visualization, offering only basic monitoring without detailed predictive insights. Thus, the existing systems lack intelligent analytics, adaptability, and predictive capabilities needed to support efficient, safe, and proactive smart building monitoring and management.

3.2 Disadvantages

- **Limited Prediction :** Existing systems mainly display current sensor values without predicting future environmental or operational conditions.
- **Threshold Dependency :** Conventional monitoring methods depend on predefined threshold values to identify abnormal conditions.
- **Limited Adaptability :** Systems cannot easily adapt to changing building conditions or emerging operational patterns.
- **Data Utilization Issues :** Historical sensor data is often not effectively utilized for predictive analysis and decision-making.

- **Delayed Detection :** Manual monitoring and basic alerts may delay the identification of abnormal environmental conditions.
- **Poor Analytical Insights :** Provides basic monitoring information without meaningful predictive insights or detailed explanations.
- **Integration Issues :** Traditional architectures provide limited integration between IoT, AI, and Digital Twin technologies.
- **Limited Scalability :** Existing systems may face difficulties supporting multiple rooms, sensors, and expanding building monitoring requirements.

3.3 System Ojectives

The major objectives of the proposed system are centered on developing an intelligent, accurate, and scalable framework for building monitoring and predictive management. The first objective is to develop an IoT-based data acquisition system using ESP32 and environmental sensors capable of collecting temperature, humidity, gas, and occupancy information from different building spaces in real time.The system focuses on continuously collecting sensor data, and variations in these parameters can be analyzed to identify abnormal conditions, making the system effective for intelligent monitoring. A second key objective is to create a robust data processing pipeline that prepares sensor datasets for analysis and deployment. This involves collecting, organizing, cleaning, and storing sensor data while ensuring that the information is suitable for real-time processing.The third objective is to design an AI-based analytics workflow that can analyze live and historical sensor data with minimal latency. This enables abnormal conditions to be identified and predictive insights to be generated for proactive building management. Another major objective is integrating Digital Twin technology with IoT and AI capabilities. By connecting real-time sensor information with a virtual representation of building spaces, the system provides a centralized monitoring environment for administrators.The project also aims to deliver a dashboard interface that allows users to monitor room conditions, visualize sensor trends, identify abnormal patterns, and receive intelligent alerts. Finally, the system is designed to be scalable and adaptable, supporting multiple rooms and sensors while improving safety, energy efficiency, maintenance planning, and data-driven decision-making in smart buildings.

3.4 Proposed Features & Functionalities

The proposed system integrates multiple modules that collectively enable intelligent building monitoring, predictive analysis, and Digital Twin visualization. The major features include:

3.4.1 IoT-Based Data Acquisition Module

Collects temperature, humidity, gas, and motion data from different building spaces using ESP32 and connected sensors. Captures real-time environmental and occupancy information for further processing.

3.4.2 Data Preprocessing Module

Processes the collected sensor data before analysis. Handles data organization, validation, cleaning, and preparation of sensor readings to ensure reliable information for real-time monitoring and AI-based analysis.

3.4.3 AI/ML Prediction Module

Analyzes historical and real-time sensor data using AI/ML techniques. Identifies patterns in building conditions and generates predictive insights to support proactive monitoring and management.

3.4.4. Anomaly Detection Module

Identifies unusual variations in temperature, humidity, gas, and occupancy conditions. Detects abnormal patterns and supports early identification of potentially unsafe or irregular building conditions.

3.4.5. Digital Twin Visualization Module

Provides a virtual representation of building spaces through an interactive dashboard. Displays room conditions and sensor information, enabling users to understand the current state of the monitored building.

3.4.6. Real-Time Monitoring and Alert Module

Continuously monitors incoming sensor data and provides alerts when abnormal conditions are identified. Enables users to respond quickly to environmental changes and potential safety issues.

3.4.7. Analytics and Reporting Module

Presents sensor readings, historical trends, analytical results, and predictive information through

graphical visualizations. Supports users in understanding building conditions and making data-driven decisions.

3.4.8. System Integration and Scalability Module

Integrates ESP32 sensors, communication services, FastAPI backend, MongoDB database, AI/ML analytics, and the Digital Twin dashboard. The architecture supports multiple rooms, sensors, and future expansion of intelligent building management.

3.5 System Architecture

The proposed system architecture integrates IoT-based sensing, AI/ML analytics, and Digital Twin technology to provide intelligent and reliable building monitoring. At its foundation, environmental and occupancy data are collected from different building spaces using ESP32-based sensors. The sensors capture parameters such as temperature, humidity, gas levels, and motion information at regular intervals.The collected sensor data is transmitted through Wi-Fi to the FastAPI backend, where it is validated, processed, and stored in the MongoDB database. This centralized storage enables the system to maintain both real-time and historical sensor information for further analysis.The processed data is then provided to the AI/ML analytics module. The module analyzes historical and real-time sensor readings to identify abnormal patterns and generate predictive insights. Detected anomalies can be used to provide intelligent alerts and support proactive building management.To provide effective visualization, the architecture incorporates a Digital Twin dashboard developed using React and Material UI. The dashboard represents monitored building spaces and displays their current sensor conditions, room status, historical trends, and analytical information.The system also includes real-time monitoring and alert functionality, enabling users to observe changing building conditions and respond to abnormal situations. The integrated architecture connects sensing, communication, storage, analytics, and visualization components into a centralized monitoring framework.Furthermore, the architecture is designed to support multiple rooms and sensors, allowing the system to be extended for larger building environments. This integrated approach enables continuous monitoring, anomaly detection, predictive

analysis, improved safety, and data-driven building management.

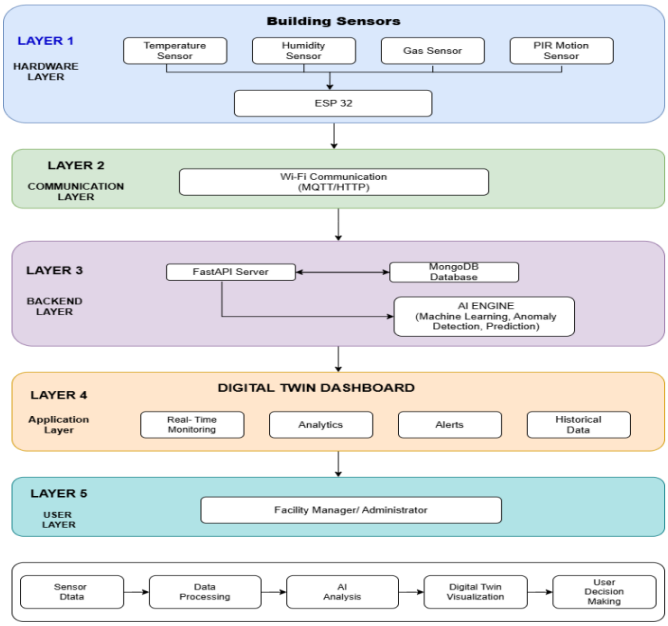


Fig 3.5.1. Architecture Model

3.6 Advantages of the Proposed System

- **Improved Efficiency:** Automates building monitoring tasks, reducing time and effort required for data collection and analysis.
- **Enhanced Accuracy:** Provides consistent sensor readings and analytical results, reducing errors during monitoring and decision-making.
- **User-Friendly Interface:** Provides an interactive dashboard that allows users to easily understand building conditions.
- **Real-Time Processing:** Enables immediate monitoring of sensor readings and changing room conditions.
- **Scalability:** Supports multiple rooms and sensors without significantly affecting system performance.
- **Cost-Effective:** Reduces manual monitoring efforts and supports efficient utilization of building resources.
- **Data Management:** Provides organized storage, retrieval, and management of real-time and historical sensor data.
- **Intelligent Analysis:** Uses AI/ML techniques to analyze sensor patterns and generate useful predictive insights.

- **Adaptability:** Can accommodate changing building conditions and expanding monitoring requirements.
- **Transparency:** Provides dashboards and visualizations for monitoring room conditions, trends, and analytical results.
- **Anomaly Detection:** Identifies unusual environmental conditions and supports early detection of potential issues.
- **Integration Capability:** Integrates IoT sensors, FastAPI, MongoDB, AI/ML analytics, and Digital Twin visualization.
- **Reliability:** Provides continuous monitoring and consistent access to building information.
- **Predictive Management:** Supports proactive maintenance and decision-making using historical and real-time sensor information.
- **Continuous Monitoring:** Continuously tracks environmental and occupancy conditions and provides alerts for abnormal situations.

CHAPTER 4

REQUIREMENT SPECIFICATION

4.1 Hardware Requirements

The proposed AI-powered Digital Twin system requires suitable hardware components for collecting and transmitting real-time building data. The ESP32 microcontroller serves as the primary processing and communication unit, providing Wi-Fi connectivity and interfacing with multiple sensors.A DHT22 sensor is used to measure temperature and humidity conditions within the monitored spaces. Gas sensors such as MQ-series sensors are used to detect changes in gas concentration and identify potentially unsafe environmental conditions. A PIR motion sensor is used to detect movement and provide occupancy-related information.Additional hardware components include breadboards, jumper wires, USB cables, and suitable power supplies for connecting and operating the sensors. The hardware arrangement can be deployed in different building spaces such as reception areas, manager cabins, and workspaces.A computer or server system is required to run the FastAPI backend, MongoDB database, AI/ML processing components, and React-based dashboard during development and deployment. The system also requires a stable Wi-Fi network for transmitting sensor readings from ESP32 devices to the backend server.The selected hardware provides a low-cost and flexible foundation for real-time environmental monitoring while allowing additional sensors and rooms to be incorporated as the system expands.

4.2 Software Requirements

The proposed system requires software components for sensor communication, backend processing, database management, AI/ML analysis, and Digital Twin visualization. The development environment supports Windows-based systems for implementation and testing.Python 3.11 or later is used for backend development and AI/ML processing. FastAPI provides the backend framework for developing RESTful APIs and handling communication between the IoT devices, database, and frontend application. MongoDB is used for storing real-time and historical sensor information.The frontend application is developed using React.js and Material UI to provide an interactive monitoring dashboard. Axios is used for communication between the frontend and backend services. AI/ML development uses libraries such as Scikit-learn, Pandas, and NumPy for data processing, analysis, and model development.Additional software tools include Visual Studio Code for development, Postman for API testing, and Git/GitHub for version control and project

management. The system may also incorporate visualization libraries such as Chart.js or Recharts for presenting sensor trends and analytical results.

4.3 Development Environment

The development environment provides the tools and platforms required to design, implement, test, and maintain the proposed Digital Twin system. Windows 10 or Windows 11 can be used as the primary operating system for development and testing.Visual Studio Code is used as the main integrated development environment for writing and managing Python, React.js, and related project files. Python 3.11 or later provides the runtime environment for FastAPI services, data processing, and AI/ML development.The backend environment consists of FastAPI and its supporting Python packages for creating RESTful APIs. MongoDB provides database services for storing sensor readings and building information. Node.js and npm support the development and execution of the React.js frontend application.Postman is used to test API requests and verify communication between system components. Git and GitHub support source-code management, version tracking, and collaborative development. This development environment provides the required tools for integrating IoT devices, backend services, database operations, AI/ML processing, and Digital Twin visualization.

4.4 Technology Stack

The proposed system uses an integrated technology stack covering IoT, backend services, database management, artificial intelligence, and frontend visualization. At the hardware level, ESP32 is used as the primary IoT controller, while DHT22, gas, and PIR sensors collect environmental and occupancy information.Python and FastAPI are used for backend development and REST API implementation. MongoDB provides flexible storage for real-time and historical sensor data. The frontend is developed using React.js and Material UI to create the Digital Twin monitoring dashboard.Axios facilitates communication between the frontend and backend services. Pandas and NumPy support data processing, while Scikit-learn provides machine learning capabilities for analytical and predictive tasks. Chart.js or Recharts can be used for displaying sensor trends and analytical results.Postman supports API testing, while Git and GitHub provide version control and project management. The combination of these technologies enables communication between IoT devices, data storage, AI/ML analysis, and Digital Twin visualization.

4.5 Functional Requirements

The proposed system shall provide the following functional requirements:

- The system shall collect temperature, humidity, gas, and motion data from monitored building spaces using ESP32 and connected sensors.
- The system shall transmit sensor readings through Wi-Fi to the backend server.
- The system shall validate, process, and store sensor data in MongoDB.
- The system shall provide REST APIs for communication between IoT devices, backend services, database, and frontend.
- The system shall display real-time sensor information and room status through the Digital Twin dashboard.
- The system shall analyze historical and real-time sensor data using AI/ML techniques.
- The system shall identify abnormal sensor patterns and environmental conditions.
- The system shall generate predictive insights based on available sensor information.
- The system shall provide alerts for detected abnormal conditions.
- The system shall display historical trends and analytical information through suitable visualizations.
- The system shall support monitoring of multiple rooms and sensors.
- The system shall provide centralized access to building monitoring information.

4.6 Non-Functional Requirements

The proposed system shall satisfy the following non-functional requirements:

- Performance: The system should process incoming sensor data efficiently and provide timely dashboard updates.
- Reliability: The system should maintain consistent monitoring and data storage during normal operation.
- Scalability: The architecture should support additional rooms, sensors, and building spaces as monitoring requirements increase.
- Usability: The dashboard should provide a simple and understandable interface for monitoring building conditions.

- Maintainability: The modular architecture should allow individual components to be modified or enhanced without affecting the complete system.
- Availability: The monitoring platform should remain accessible whenever the required network and server services are operational.
- Security: Communication and stored building data should be protected through appropriate authentication and access-control mechanisms.
- Accuracy: Sensor readings and analytical results should be processed consistently to support reliable monitoring and decision-making.
- Interoperability: The system should allow communication between IoT devices, backend APIs, databases, AI/ML modules, and frontend applications.
- Extensibility: The system should allow future integration of additional sensors, AI models, visualization features, and building management capabilities.

CHAPTER 5
SYSTEM DESIGN

5.1 Proposed Model

The proposed AI-powered Digital Twin system is designed to provide intelligent building monitoring and predictive management in real-time. It follows a modular architecture that integrates the IoT sensing layer, backend services, database, AI/ML analytics, and Digital Twin dashboard, simplifying maintenance, improving scalability, and allowing individual components to be enhanced without disrupting the complete system. The IoT layer focuses on collecting environmental and occupancy information using ESP32 and connected sensors, while the backend manages data processing, API communication, and secure storage of sensor information.The system emphasizes real-time processing of building conditions, enabling abnormal environmental patterns to be identified and analyzed promptly. AI/ML techniques are used to analyze historical and real-time sensor data, generate predictive insights, and support proactive building management. Scalability is an important design principle, allowing the system to support increasing numbers of rooms, sensors, and building spaces without compromising monitoring performance.A Digital Twin dashboard provides users with real-time visualization of room conditions, sensor values, trends, and analytical information. The modular architecture also supports integration of alerts and anomaly detection for improving building safety and maintenance planning. The system is designed to be flexible and extensible, allowing additional sensors, analytical models, and visualization capabilities to be incorporated as future requirements evolve.

5.2 System Architecture Design

The system architecture is designed to integrate IoT-based sensing, data communication, backend processing, database management, AI/ML analytics, and Digital Twin visualization into a unified building monitoring framework. The architecture consists of interconnected layers that enable continuous collection, processing, analysis, storage, and visualization of building information.At the sensing layer, ESP32 acts as the primary controller and collects temperature, humidity, gas, and motion data from sensors installed in different building spaces. The collected readings are transmitted through Wi-Fi to the backend system. This communication layer enables the continuous transfer of sensor

information for real-time monitoring and further analysis.The backend layer is implemented using FastAPI and provides RESTful APIs for receiving sensor data, managing building and room information, and communicating with the database and frontend application. MongoDB is used as the database layer for storing real-time and historical sensor information. The stored data provides a foundation for monitoring, analysis, and future predictive tasks.The AI/ML layer processes available sensor data to identify abnormal patterns and generate predictive insights. Historical and real-time information can be analyzed to support anomaly detection, proactive monitoring, and intelligent decision-making. The Digital Twin layer provides a virtual representation of the monitored building spaces through a React-based dashboard.The presentation layer displays room status, sensor values, historical trends, and analytical information to users. This layered architecture provides modularity, scalability, and flexibility, allowing additional rooms, sensors, analytical models, and visualization features to be integrated as the system develops.

5.3 IoT Sensor and Data Collection Design

The IoT sensor and data collection design forms the foundation of the proposed Digital Twin system by continuously capturing environmental and occupancy information from different building spaces. The system uses an ESP32 microcontroller as the primary sensing and communication unit. Sensors are connected to the ESP32 to collect temperature, humidity, gas, and motion information from monitored areas such as the reception, manager cabin, and workspace.The DHT22 sensor is used to measure temperature and humidity, while the gas sensor detects changes in gas concentration within the monitored space. A PIR motion sensor is used to identify movement and provide occupancy-related information. Each sensor reading is associated with a room identifier and timestamp so that the collected information can be accurately related to a specific building space and time.The ESP32 reads the sensor values at regular intervals and prepares the collected information as structured data. The data includes room ID, temperature, humidity, gas level, motion status, and timestamp. The ESP32 uses Wi-Fi connectivity to transmit the collected readings to the backend through HTTP requests.The FastAPI backend receives the sensor information through REST APIs and processes the incoming data before storing it in MongoDB. This enables real-time sensor information to be maintained along with historical readings for subsequent monitoring and analysis.The designed data collection mechanism provides a continuous flow of building information from physical spaces to the Digital Twin system. It also supports the addition of further rooms and sensors, providing a scalable foundation for intelligent building monitoring and future AI/ML-based analysis.

5.4 Data Preprocessing

Data preprocessing is an important stage in the proposed system because the quality of sensor data directly affects the reliability of monitoring and AI/ML analysis. The sensor readings collected from different building spaces are first received by the backend and organized according to room identification and timestamp information.The collected data is examined to identify missing, invalid, or inconsistent sensor readings. Temperature, humidity, gas, and motion values are validated to ensure that the recorded information is suitable for further processing. Unnecessary or duplicate records can be identified and removed to maintain the quality of the dataset.The sensor parameters are organized into a consistent structure containing room ID, timestamp, temperature, humidity, gas level, motion status, and related sensor information. Numerical sensor values can be normalized or scaled when required for machine learning algorithms so that parameters with different ranges can be processed effectively.The preprocessed data is stored in MongoDB and made available for real-time monitoring and subsequent AI/ML analysis. Historical sensor information can also be prepared for identifying patterns and developing predictive models. The preprocessing stage therefore provides reliable and structured input for anomaly detection, predictive analytics, and Digital Twin visualization.This process improves the consistency of sensor information and helps reduce the effect of invalid or incomplete readings on subsequent analysis. It also establishes a standardized data format that supports the integration of multiple rooms and sensors within the proposed intelligent building monitoring system.

5.5 AI/ML Model Design

The AI/ML model design forms an intelligent layer of the proposed Digital Twin system for analyzing sensor data and supporting predictive building management. The model uses historical and real-time sensor information such as temperature, humidity, gas level, and occupancy conditions as inputs for analysis.The collected and preprocessed sensor data is organized according to room and timestamp information to identify patterns in building conditions. Relevant features are selected from the available sensor parameters and provided to the machine learning process. The model learns relationships and patterns within the sensor data that can be used to understand normal building behavior and identify changes in environmental conditions.The AI/ML component is designed to analyze incoming sensor readings and compare them with previously observed patterns. When significant deviations or unusual combinations of sensor values are identified, the system can provide information for further anomaly

analysis. Historical data can also be used to improve the model and support prediction of future building conditions.The model output is integrated with the backend and Digital Twin dashboard so that analytical results can be presented to users along with real-time sensor information. This enables users to understand current building conditions and obtain data-driven insights for proactive monitoring and management.The AI/ML design is modular, allowing different machine learning algorithms and additional features to be incorporated as the dataset grows. This provides flexibility for improving predictive performance and extending the system toward intelligent maintenance, energy management, and automated building decision support.

5.6 Anomaly Detection Design

The anomaly detection design enables the proposed system to identify unusual conditions in the monitored building spaces by analyzing sensor readings and their variations over time. The detection process uses parameters such as temperature, humidity, gas level, and occupancy information obtained from the IoT sensing layer.The preprocessed sensor data is analyzed to establish normal patterns for different rooms and environmental conditions. Incoming sensor readings are then compared with the learned patterns to identify significant deviations. When a sensor value or combination of values differs considerably from the expected behavior, the condition can be identified as an anomaly.The detected anomalies are associated with the corresponding room and timestamp to provide meaningful information about where and when the unusual condition occurred. This information is passed to the backend and made available to the monitoring dashboard for further observation.The anomaly detection mechanism can support the identification of abnormal environmental conditions, unexpected gas-level changes, unusual temperature or humidity variations, and changes in occupancy patterns. The results can also be combined with the AI/ML analytics module to provide predictive insights and support proactive decision-making.The detected conditions are displayed through the Digital Twin dashboard, allowing users to observe abnormal room conditions alongside real-time sensor information. Intelligent alerts can also be generated when important deviations are identified.This design helps reduce dependence on continuous manual observation and supports early identification of potentially unsafe or irregular conditions. The modular approach also allows the anomaly detection method to be improved as additional historical sensor data becomes available.

5.7 Digital Twin Design

The Digital Twin design provides a virtual representation of the monitored building spaces and connects the physical environment with real-time sensor information. The design enables users to observe the current condition of different rooms through an interactive digital interface.The Digital Twin receives processed information from the backend, including room identification, temperature, humidity, gas level, motion status, and timestamp. These values are associated with their respective building spaces so that the virtual representation reflects the current state of the physical environment.The dashboard is developed using React.js and Material UI to provide an interactive and user-friendly monitoring interface. Each monitored room can display its current status and corresponding sensor information. The Digital Twin also presents historical sensor trends and analytical information to help users understand changes in building conditions over time.The Digital Twin design integrates real-time monitoring with AI/ML-based analysis and anomaly detection. When abnormal conditions are identified, the corresponding room information can be highlighted through the monitoring interface and associated alerts can be provided to the user.The design also supports centralized monitoring of multiple building spaces from a single dashboard. This reduces the need for manual observation of individual rooms and provides users with a consolidated view of building conditions.The modular Digital Twin design allows additional rooms, sensors, visualization components, and analytical features to be incorporated in the future. Thus, it provides a flexible interface for representing physical building conditions digitally while supporting intelligent monitoring and predictive building management.

5.8 Database Design

The database design provides a structured mechanism for storing and managing the information generated by the proposed Digital Twin system. MongoDB is used as the primary database because it supports flexible storage of sensor information and allows data to be organized according to different building spaces and monitoring requirements. The database stores information collected from the ESP32-based sensing units, including room ID, temperature, humidity, gas level, motion status, and timestamp. Building and room information is also maintained so that sensor readings can be associated with their respective physical spaces. Each sensor record contains the required information for identifying the source and time of the measurement. This organization allows the system to retrieve recent readings for real-time monitoring and historical records for analysis. Historical data can be used

by the AI/ML module for identifying patterns, detecting anomalies, and generating predictive insights. The FastAPI backend provides the interface between the database and other system components. REST APIs are used to receive sensor information, store records, retrieve building and room details, and provide data to the React-based Digital Twin dashboard. The database design supports continuous insertion of sensor readings while allowing users to retrieve relevant information based on room, sensor parameter, or time. This enables efficient presentation of current conditions and historical trends through the dashboard. The flexible database structure also supports system expansion by allowing additional rooms, sensors, and data attributes to be incorporated as future monitoring requirements increase. Thus, the database provides a centralized foundation for reliable storage, retrieval, and analysis of building monitoring information.

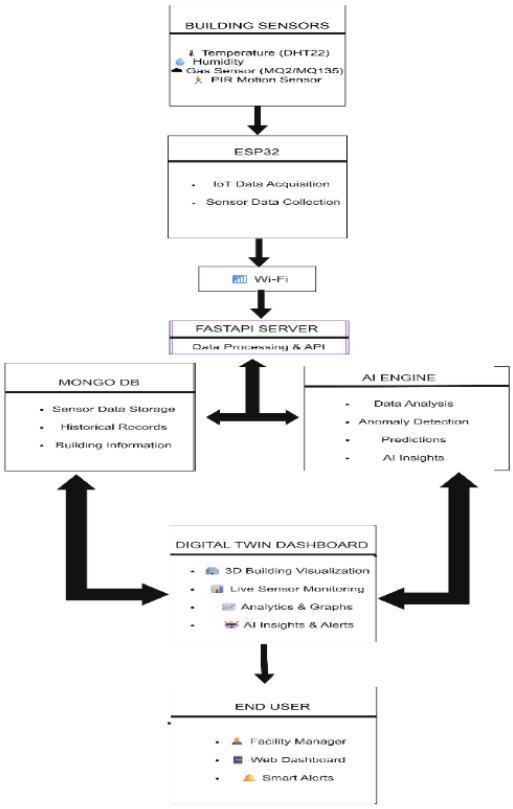


Fig 5.8.1. End-to-End Workflow of the Platform

5.9 Dashboard Design

The dashboard design provides a centralized interface for monitoring and analyzing the conditions of different building spaces. It connects the Digital Twin visualization with the backend services to present real-time sensor information and analytical results in an accessible format. The dashboard is developed using React.js and Material UI, providing a structured and interactive user interface. Axios

is used to communicate with the FastAPI backend and retrieve the required building, room, and sensor information. The interface displays parameters such as temperature, humidity, gas level, and motion status for the monitored spaces. The dashboard provides room-wise monitoring so that users can identify the current condition of individual building areas. Sensor readings are presented using suitable cards, status indicators, and graphical visualizations. Historical sensor information can also be displayed as trends to help users understand changes in environmental conditions over time. The dashboard integrates information from the AI/ML and anomaly detection modules. When unusual conditions are detected, the relevant room and sensor information can be presented to users through alerts or status indicators. Predictive and analytical results can also be incorporated to support data-driven building management. The interface is designed to provide a simple and understandable monitoring experience while maintaining access to important building information. The centralized dashboard reduces the need to monitor individual sensors separately and provides a consolidated view of the building. The modular dashboard design allows additional rooms, sensor parameters, charts, alerts, and analytical features to be incorporated in future development. Thus, it serves as the primary user interface for interacting with the Digital Twin and monitoring the intelligent building environment.

5.10 Deployment Design

The deployment design defines how the proposed AI-powered Digital Twin system is operated as an integrated monitoring platform. The ESP32-based sensing units are deployed in different building spaces and connected to the required environmental and occupancy sensors. A stable Wi-Fi network is used to transmit the collected sensor information to the backend services. The FastAPI backend is responsible for receiving sensor data, processing requests, and communicating with the MongoDB database. The database maintains real-time and historical building information required for monitoring and analysis. The React.js application provides the user-facing Digital Twin dashboard and retrieves the required information through REST APIs. During deployment, the IoT devices, backend, database, AI/ML components, and dashboard are configured to communicate with each other through the defined interfaces. The system can be tested initially in a controlled prototype environment before being extended to additional building spaces. The deployment design supports continuous sensor data collection and real-time monitoring while maintaining access to historical information for AI/ML analysis and anomaly detection. The dashboard can be accessed through a compatible web browser, allowing users to observe room conditions and analytical information from a centralized interface. The modular deployment approach allows individual components to be updated or expanded without

requiring complete system replacement. Additional ESP32 devices, sensors, rooms, and analytical capabilities can therefore be incorporated as monitoring requirements increase. This deployment design provides a practical foundation for implementing the proposed Digital Twin system in smart building environments while supporting future improvements in predictive analytics, intelligent alerts, energy management, and automated building operations.

CHAPTER 6
IMPLEMENTATION

6.1 Overview

The implementation of the proposed AI-powered Digital Twin system focuses on integrating IoT hardware, backend services, database management, AI/ML processing, and Digital Twin visualization into a functional building monitoring platform. The implementation follows the system design described in the previous chapter and establishes communication between the physical building environment and its virtual representation.The implementation begins with the ESP32 microcontroller and connected sensors. The DHT22 sensor collects temperature and humidity values, while the gas sensor measures changes in gas concentration and the PIR sensor detects motion within the monitored building spaces. The ESP32 reads these sensor values at regular intervals and transmits the collected information through Wi-Fi to the backend server.The backend is implemented using Python and FastAPI. REST APIs are developed to receive sensor readings, process incoming requests, and communicate with the MongoDB database. The database stores real-time and historical sensor information along with room identification and timestamps. This stored data is used for monitoring, visualization, and AI/ML analysis.The AI/ML component processes the prepared sensor data to identify patterns, detect abnormal conditions, and generate predictive insights. The processed results are integrated with the backend so that relevant analytical information can be provided to the user interface.The Digital Twin dashboard is implemented using React.js and Material UI. It provides a visual representation of monitored building spaces and displays real-time sensor values, room conditions, historical trends, anomaly information, and system alerts. The frontend communicates with the FastAPI backend using API requests to obtain updated information.The complete implementation connects the physical sensing layer with the digital representation through continuous data flow. This enables real-time monitoring, intelligent analysis, anomaly identification, and data-driven building management. The modular implementation also allows additional sensors, rooms, AI models, and dashboard features to be integrated in future development.

6.2 Hardware Implementation

The hardware implementation forms the physical sensing layer of the proposed AI-powered Digital Twin system. It is responsible for collecting real-time environmental and occupancy-related information from different spaces within the building. The hardware setup consists of an ESP32 microcontroller, DHT22 temperature and humidity sensor, gas sensor, and PIR motion sensor. These components are deployed in the monitored building spaces, namely the Reception, Manager Cabin, and Workspace. The ESP32 acts as the central processing and communication unit of each sensing setup. It interfaces with the connected sensors, reads their outputs at regular intervals, and prepares the collected values for transmission to the backend server. The ESP32 also provides built-in Wi-Fi connectivity, allowing the physical sensing layer to communicate directly with the software system. These parameters provide information about the environmental conditions of each monitored space and are used for real-time monitoring and subsequent analytical processing. The gas sensor is used to identify changes in gas concentration within the monitored area. The PIR sensor detects motion and provides information related to activity or occupancy conditions in the corresponding space. The collected sensor values are associated with the respective room identifier and timestamp. The ESP32 organizes these readings into a structured data format before transmitting them to the backend through the communication layer. This enables consistent processing and storage of sensor information. The hardware layer therefore establishes the connection between the physical building environment and the digital system. Continuous sensor measurements provide the input required for real-time monitoring, historical analysis, AI/ML processing, anomaly identification of the Digital Twin with the actual building conditions.

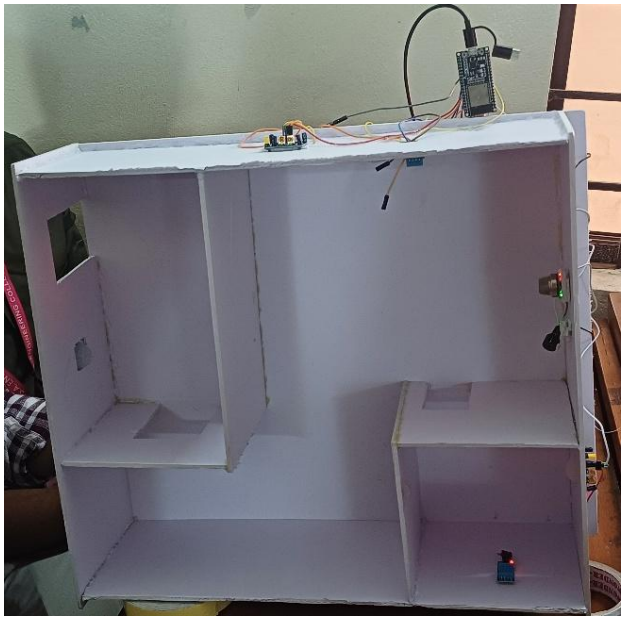


Fig 6.2.1. Hardware setup

6.3 IoT Data Acquisition and Communication

The IoT data acquisition and communication layer is responsible for collecting sensor measurements from the monitored building spaces and transferring the data to the backend system. The ESP32 continuously reads the values generated by the connected DHT22, gas, and PIR sensors. Each reading represents the current environmental or activity condition of the corresponding building space. The DHT22 sensor provides temperature and humidity measurements, while the gas sensor provides the gas-level reading. The PIR sensor provides the motion status of the monitored space. The ESP32 processes these sensor outputs and combines them with the corresponding room identifier and timestamp to form a structured sensor-data record. After collecting the sensor readings, the ESP32 establishes communication with the backend server through a Wi-Fi network. The collected information is formatted as a JSON payload and transmitted to the FastAPI server using an HTTP POST request. The payload contains the room identifier, sensor measurements, motion status, and timestamp. This structured format allows the backend to receive and process data consistently from different monitored spaces. The communication process is performed continuously at defined intervals so that updated sensor information can be made available to the software system. The backend receives the transmitted payload, validates the incoming information, and forwards the required data for storage and further processing. This enables the system to maintain a continuous data flow between the physical building and the digital platform. The IoT communication layer therefore provides the link between the physical sensing environment and the backend services. The continuous transmission of sensor data supports real-time monitoring, historical data collection, AI/ML analysis, anomaly detection, and synchronization of the Digital Twin with the current state of the monitored building spaces.

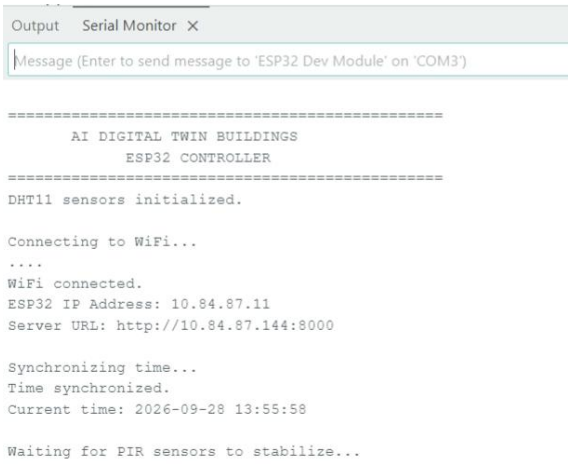


Fig 6.3.1. Hardware Communication

6.4 Backend Implementation

The backend implementation provides the communication and processing layer between the IoT sensing devices, database, AI/ML modules, and frontend dashboard. The backend is developed using Python and the FastAPI framework. It receives sensor data transmitted by the ESP32 devices, validates the incoming information, manages database operations, and provides APIs for retrieving data required by the Digital Twin and dashboard. FastAPI is used to implement the RESTful API layer because it provides a lightweight and efficient framework for handling HTTP-based communication. The backend exposes API endpoints through which the ESP32 devices can submit sensor readings and the frontend can request stored or processed information. The API-based architecture separates the sensing, processing, storage, and visualization components, making the system easier to maintain and extend. When an ESP32 device sends a sensor reading, the backend receives the request through the corresponding REST endpoint. The request contains structured information such as the room identifier, temperature, humidity, gas level, motion status, and timestamp. The backend parses the incoming JSON payload and processes the received values before storing them in the database. Input validation is performed before database insertion to ensure that the received data follows the expected structure and data types. The backend verifies the required fields and checks whether the sensor values can be processed correctly. This validation prevents malformed or incomplete requests from affecting the stored dataset and improves the reliability of subsequent analytics and AI/ML processing. After successful validation, the backend establishes communication with the MongoDB database and stores the sensor record. Each record is associated with the corresponding room and timestamp, allowing the system to maintain both current and historical sensor information. The backend also provides data retrieval operations that allow the frontend and analytical modules to access the required records from MongoDB. The backend acts as an intermediate layer between the database and the application interface. When the dashboard requests sensor information, the FastAPI server retrieves the required records from MongoDB and returns the information through API responses. Similarly, AI/ML processing can use the stored sensor data through the backend data-access layer, while the resulting analytical information can be made available to the frontend. The implemented backend therefore maintains the continuous flow of information across the system. It connects the ESP32-based sensing layer with MongoDB, AI/ML processing, and the Digital Twin dashboard. This architecture enables reliable sensor-data handling, centralized data management,

and integration of real-time monitoring with intelligent building analytics.

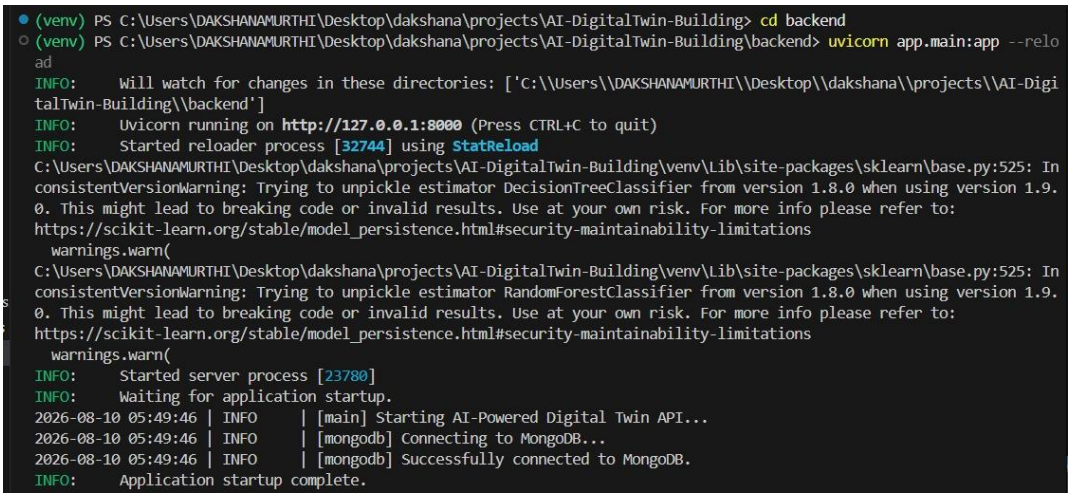


Fig 6.4.1. Backend Connectivity

6.5 Database Implementation

The database layer is implemented using MongoDB to store and manage the sensor data generated by the IoT devices. MongoDB is used as the central data storage system because the sensor readings are generated continuously and contain multiple fields related to different building spaces. The database maintains the sensor information required for real-time monitoring, historical analysis, and AI/ML processing. The FastAPI backend communicates with MongoDB whenever new sensor data is received from the ESP32. After the incoming request is validated, the sensor record is stored in the database along with its corresponding room identifier and timestamp. The stored record contains the measured temperature, humidity, gas level, and motion status. This provides a structured historical record of the conditions observed in each monitored space. The database maintains data for the different monitored building spaces, including the Reception, Manager Cabin, and Workspace. The room identifier allows the backend to distinguish readings belonging to different spaces and retrieve room-specific information when required. Timestamps are also stored with the sensor readings so that the system can identify when each measurement was collected and perform time-based analysis. MongoDB also supports retrieval of historical sensor records required by the application. The backend can query the stored information based on parameters such as room and time period and provide the retrieved data to the frontend and analytical modules. Historical records can therefore be used to display sensor trends and provide input for AI/ML processing. The database acts as the persistent storage layer of the proposed system. Real-time sensor readings are continuously added to the database, while previously stored records remain available for analysis and visualization. This separation between data storage and application processing allows the system to maintain historical information without depending only on the current sensor state.

The integration of MongoDB with FastAPI provides a continuous data-management pipeline from sensor acquisition to storage and retrieval. The stored data serves as a common source for the monitoring dashboard, Digital Twin visualization, historical analysis, and AI/ML-based processing, supporting the overall functionality of the intelligent building monitoring system.

6.6 Data Preprocessing Implementation

The data preprocessing stage prepares the sensor data stored in MongoDB for further analysis and AI/ML processing. Since the system continuously collects data from multiple sensors and building spaces, the raw sensor readings need to be organized into a consistent format before they are used by the analytical components. The preprocessing process begins by retrieving the required sensor records from the database through the backend. The collected records contain the room identifier, temperature, humidity, gas level, motion status, and timestamp. These fields are organized so that the sensor parameters can be processed consistently across the monitored spaces. The received data is checked for missing, incomplete, or invalid values before being passed to the AI/ML module. Data validation helps reduce the effect of incorrect sensor readings and ensures that the input provided to the analytical models follows the expected structure. The timestamp and room identifier are also retained so that the processed data can be associated with the correct building space and time period. The relevant sensor parameters are then selected as features for analytical processing. Temperature, humidity, gas level, and motion-related information provide the primary input variables for identifying environmental patterns and abnormal conditions. The prepared data is arranged into a suitable feature structure for use by the implemented AI/ML models. The preprocessing stage also maintains the relationship between sensor readings and their corresponding rooms. This allows the system to perform room-specific analysis and relate prediction or anomaly results to the appropriate space in the Digital Twin. The processed information can then be supplied to the AI/ML module for model-based analysis. Thus, the data preprocessing implementation provides a reliable transition between raw IoT sensor data and AI/ML processing. It improves the consistency of the input data and supports subsequent prediction, anomaly detection, and analytical operations within the proposed intelligent building monitoring system.

6.7 AI/ML Implementation

The AI/ML implementation forms the intelligent processing layer of the proposed Digital Twin system. The purpose of this layer is to analyze the collected building sensor data and generate useful insights beyond conventional real-time monitoring. The processed temperature, humidity, gas-level, and motion-

related data are used as inputs for the implemented machine learning processes. The AI/ML module receives the prepared sensor data from the data preprocessing stage. The relevant sensor parameters are organized into feature inputs, while the associated timestamps and room information are maintained to support building-space-specific analysis. Historical sensor records stored in the database provide the data required for developing and evaluating the machine learning models. The implementation includes model training using the prepared sensor dataset. During training, the models learn patterns and relationships present in the historical building data. The trained models are then used to process new sensor observations and generate predictive results. This allows the system to move from simple observation of current sensor values toward data-driven prediction of building conditions. The AI/ML component is also integrated with the anomaly detection process to identify sensor conditions that differ significantly from expected patterns. Such abnormal conditions can be associated with environmental or occupancy-related changes within the monitored spaces. The resulting analytical information is passed to the backend and made available to the Digital Twin dashboard. The implemented AI/ML functionality supports predictive analytics related to building operation. It provides a foundation for predicting relevant building conditions, identifying abnormal behavior, and supporting predictive maintenance and intelligent decision-making. The AI-generated results are combined with real-time sensor information so that the user can view both the current physical state and the corresponding analytical insights. The integration of AI/ML with the Digital Twin improves the system from a reactive monitoring platform to a more proactive intelligent building-management system. Instead of relying only on current sensor readings, the system uses historical and real-time data to identify patterns, detect abnormal conditions, and generate predictive information for the monitored building spaces.

6.8 Anomaly Detection Implementation

The anomaly detection component is implemented to identify unusual conditions in the monitored building spaces using the collected sensor data. The purpose of this component is to distinguish normal building conditions from sensor patterns that deviate from the expected behavior. This provides an additional layer of intelligence to the Digital Twin beyond conventional real-time monitoring. The anomaly detection process uses the preprocessed sensor readings obtained from the building monitoring system. Parameters such as temperature, humidity, gas level, and motion status are analyzed to identify abnormal changes or patterns. The room identifier is retained during the analysis so that any detected abnormal condition can be associated with the corresponding building space. The system evaluates incoming sensor observations against the learned or defined normal behavior of the monitored

environment. When a sensor reading or combination of readings shows an abnormal pattern, the corresponding observation is identified as an anomaly. The detected result is then processed by the backend and made available to the application interface. Anomaly information is integrated with the Digital Twin dashboard so that abnormal conditions can be distinguished from normal building states. This allows users to observe unusual environmental or activity conditions in the relevant building space without manually examining every individual sensor reading. The anomaly detection results can also support the predictive analytics component by providing information about unusual patterns observed in the historical and real-time data. These results contribute to early identification of potentially undesirable building conditions and assist in data-driven building management. The implementation of anomaly detection therefore enhances the intelligence of the proposed system by converting raw sensor observations into meaningful condition-based information. Combined with the Digital Twin visualization and AI/ML processing, it enables the system to provide a more proactive view of the building environment.

6.9 Digital Twin Implementation

The Digital Twin implementation provides a virtual representation of the monitored building spaces and connects the physical sensing environment with its digital counterpart. The Digital Twin uses the sensor information collected from the physical building to represent the current condition of each monitored space in the software environment. The implementation represents the monitored areas, namely the Reception, Manager Cabin, and Workspace, as individual building spaces within the digital environment. Each space is associated with its corresponding sensor data so that the environmental and activity conditions of the physical space can be represented in the Digital Twin. The sensor data received from the ESP32 devices is processed through the backend before being provided to the Digital Twin interface. Parameters such as temperature, humidity, gas level, and motion status are mapped to the corresponding building space. The room identifier ensures that the received measurements are displayed for the correct location. The Digital Twin is continuously updated using the latest available sensor information. As new readings are received and stored, the corresponding virtual representation reflects the updated state of the physical environment. This synchronization provides a real-time connection between the physical building and its digital representation. The Digital Twin also incorporates analytical information generated by the AI/ML and anomaly detection components. Therefore, the virtual representation is not limited to displaying raw sensor values. It can also present relevant prediction results, detected abnormal conditions, and analytical information associated with the monitored spaces. The implementation provides a unified view of the building by combining

physical sensor observations with digital visualization and intelligent analysis. This allows users to understand the current state of different building spaces and observe changes through the Digital Twin interface. The integration of real-time IoT data, historical information, and AI/ML results makes the Digital Twin a central component of the proposed intelligent building monitoring system.

6.10 Frontend and Dashboard Implementation

The frontend and dashboard provide the user interface for monitoring the building and viewing the information generated by the proposed system. The dashboard is implemented using React.js and Material UI. It communicates with the FastAPI backend through API requests and presents the sensor data, Digital Twin information, historical trends, and analytical results in an organized interface. The frontend retrieves the required information from the backend and displays it according to the corresponding building space. The monitored spaces, including the Reception, Manager Cabin, and Workspace, are presented separately so that users can observe the condition of each area. The dashboard displays the latest available temperature, humidity, gas-level, and motion information for the selected space. The dashboard also provides visualization of historical sensor data. Time-based sensor information retrieved from MongoDB through the backend can be represented using suitable charts and graphical elements. This allows users to observe changes in environmental conditions over time rather than relying only on individual real-time readings. The AI/ML and anomaly detection results are integrated into the dashboard along with the sensor information. Prediction results and detected abnormal conditions are presented as part of the building analytics interface. This enables users to interpret the current sensor state together with the analytical information generated by the system. Material UI components are used to maintain a structured and consistent interface for the different dashboard elements. The frontend is organized into reusable components so that individual monitoring and visualization elements can be maintained independently. React.js also allows the interface to update the displayed information as new data becomes available from the backend. The dashboard therefore acts as the main user-facing layer of the proposed system. It combines real-time IoT measurements, historical data, Digital Twin representation, and AI/ML-based analytical results into a unified monitoring interface. This provides users with a clear view of the building condition and supports data-driven interpretation of the monitored spaces.

6.11 System Integration

The system integration stage combines the hardware, communication, backend, database, AI/ML, Digital Twin, and frontend components into a single functional building monitoring platform. The integration establishes a continuous flow of information from the physical building environment to the digital representation and analytical interface. The process begins with the IoT sensing layer, where the ESP32 collects temperature, humidity, gas-level, and motion information from the monitored building spaces. The collected readings are formatted as structured data and transmitted through Wi-Fi to the FastAPI backend. The backend receives the sensor data through the implemented REST API endpoints and performs the required request validation before processing the data. After successful validation, the sensor records are stored in MongoDB along with the corresponding room identifier and timestamp. The stored data provides the persistent data source required for historical monitoring and AI/ML processing. The preprocessing component prepares the required sensor information before it is supplied to the analytical modules. The AI/ML component processes the prepared data to generate predictive and analytical information, while the anomaly detection component identifies unusual conditions within the monitored spaces. The resulting information is integrated with the backend so that both sensor observations and analytical results can be accessed by the frontend. The Digital Twin receives the relevant real-time sensor information and represents the current state of the corresponding physical building spaces. At the same time, the React.js and Material UI dashboard retrieves information from the backend and presents the sensor readings, Digital Twin state, historical trends, prediction results, and anomaly information to the user.

The complete integrated workflow can therefore be represented as:

Sensors → ESP32 → Wi-Fi → FastAPI → MongoDB → Data Preprocessing → AI/ML and Anomaly Detection → Digital Twin → React.js Dashboard

This integration ensures that the physical building, data-management layer, intelligent processing layer, and virtual representation operate as a connected system. The continuous flow of sensor data enables the Digital Twin and dashboard to reflect the monitored building conditions while providing analytical information for intelligent building monitoring.

6.12 Implementation Summary

The implementation of the proposed AI-powered Digital Twin system integrates IoT-based sensing, wireless communication, backend services, database management, AI/ML processing, anomaly detection, Digital Twin visualization, and a web-based dashboard into a unified building monitoring

platform. The ESP32-based sensing layer collects real-time environmental and activity-related data from the monitored building spaces and transmits the readings to the FastAPI backend through Wi-Fi communication. The backend validates and processes the incoming sensor information before storing it in MongoDB for real-time and historical analysis. The prepared sensor data is used by the AI/ML and anomaly detection components to generate predictive and analytical insights. These results are integrated with the Digital Twin so that the virtual representation can reflect the current condition of the corresponding physical spaces. The React.js and Material UI dashboard provides a centralized interface for viewing sensor measurements, Digital Twin information, historical trends, prediction results, and detected anomalies. The integration of these components establishes a continuous data flow between the physical building and its digital representation. Overall, the implemented system provides an integrated platform for real-time building monitoring and intelligent analysis. The combination of IoT sensing, data storage, AI/ML processing, anomaly detection, and Digital Twin visualization supports a more proactive and data-driven approach to building management.

CHAPTER 7

RESULTS AND DISCUSSION

7.1 Overview

The results of the proposed AI-powered Digital Twin system demonstrate the integration of IoT-based sensing, backend processing, database management, AI/ML analysis, anomaly detection, and Digital Twin visualization for intelligent building monitoring. The implemented system was evaluated based on its ability to collect sensor data, process the information, generate analytical results, and present the building conditions through the web-based dashboard. The IoT layer provides continuous measurements from the monitored building spaces using temperature, humidity, gas, and motion sensors connected to the ESP32. The collected data is transmitted to the FastAPI backend and stored in MongoDB, enabling both current and historical sensor information to be accessed by the application. The AI/ML component processes the prepared sensor data to identify patterns and generate predictive information. The anomaly detection component further analyzes the sensor conditions to identify unusual observations. These results provide additional information beyond the direct sensor measurements and support a more proactive approach to building monitoring. The Digital Twin and dashboard combine real-time sensor information with the analytical results and present them according to the corresponding building spaces. This allows users to observe the current condition of the Reception, Manager Cabin, and Workspace while also reviewing historical trends and detected abnormal conditions. The results demonstrate that the integration of IoT, AI/ML, and Digital Twin technologies can provide a unified platform for monitoring and analyzing building conditions. The following sections discuss the model performance, real-time recommendations, overall system behavior, and visualization and reporting of the generated data.

7.2 Model Performance

The proposed system incorporates multiple machine learning models to support intelligent analysis of building conditions. The implemented models are used for gas-level classification and occupancy-related prediction, allowing the system to analyze sensor information beyond basic real-time monitoring. The trained models are stored separately and integrated with the backend prediction

services. The gas classification model is implemented to classify gas-related conditions using the available sensor information. The model receives the required sensor features and produces a classification result that can be used as part of the building safety analysis. This enables the system to identify different gas-related conditions and provide additional information for evaluating the condition of a monitored space. The occupancy prediction functionality is implemented using trained occupancy models. The occupancy model provides an occupancy-related prediction together with a confidence value. The backend response structure also records the model used for the prediction, allowing the generated result to be associated with the corresponding machine learning model. A separate room occupancy model is also included to support room-level occupancy analysis. The system maintains the relationship between the prediction and the corresponding room through the room identifier. The occupancy prediction response includes the predicted occupancy count, confidence value, temperature, CO_2-related information, and PIR state. This allows occupancy estimation to be considered together with the environmental and motion information collected from the monitored space. The model outputs are integrated into the backend prediction pipeline rather than being used as isolated offline results. When the latest sensor information is processed, the relevant model generates the corresponding prediction, which is then combined with the current sensor state and made available to the application interface. In addition to the machine learning predictions, the backend maintains a rule-based risk assessment containing a risk score, detected reasons, normalized sensor features, and a suggested action. This allows the system to combine machine learning outputs with sensor-based safety assessment for a more complete interpretation of the monitored building condition. The use of multiple models therefore improves the analytical capability of the proposed system. Gas classification provides information about gas-related conditions, while the occupancy models provide occupancy-related predictions at the building-space level. The generated predictions and confidence values are integrated with the Digital Twin and dashboard to support real-time interpretation of building conditions.

7.3 Real-Time Recommendations

The proposed system provides real-time recommendations by combining the latest IoT sensor readings with the outputs generated by the implemented AI/ML models and the system's risk assessment process. The objective is to convert the monitored building conditions into meaningful information that can assist users in understanding the current state of each monitored space. The recommendation process begins when the latest sensor data is received from the ESP32 and

processed by the backend. Parameters such as temperature, humidity, gas level, and motion status are analyzed according to the requirements of the corresponding analytical models. The AI/ML component generates occupancy-related and gas-condition results, while the risk assessment process evaluates the available sensor conditions to identify potentially abnormal or undesirable situations. Based on the detected condition, the system generates a corresponding suggested action. The recommendation is associated with the monitored room so that users can identify the location requiring attention. This approach reduces the need for users to manually interpret individual sensor readings and provides a more direct understanding of the current building condition. For occupancy-related conditions, the system uses the occupancy prediction and room occupancy results to provide information about the estimated occupancy state of the monitored space. The prediction confidence is also retained so that the generated result can be interpreted together with the model's confidence level. For gas-related conditions, the gas classification result contributes to identifying the condition of the monitored environment. When the system identifies a potentially undesirable gas-related condition, the corresponding risk information and suggested action can be presented to the user through the dashboard. The recommendation mechanism also considers the risk score and detected reasons generated by the backend. These values provide additional context for understanding why a particular condition has been identified as requiring attention. The combination of prediction results, sensor information, risk assessment, and suggested actions provides a more informative monitoring process. The generated recommendations are made available through the Digital Twin and web dashboard. Users can therefore observe the current condition of the Reception, Manager Cabin, and Workspace together with the corresponding analytical information. This enables the system to support faster interpretation of building conditions and provides a foundation for proactive building management. Overall, the real-time recommendation mechanism extends the system from simple sensor monitoring to intelligent decision support. By combining continuously collected IoT data with AI/ML predictions and risk-based analysis, the proposed system provides actionable information that can assist users in responding to changing conditions within the monitored building spaces.

7.4 Discussion

The results of the proposed AI-powered Digital Twin system demonstrate the integration of real-time IoT monitoring with machine learning-based analysis for intelligent building management. The system continuously collects environmental and activity-related information from the monitored

building spaces and processes the data through the backend, database, AI/ML models, and Digital Twin interface. The implemented machine learning models provide additional analytical capabilities beyond direct sensor visualization. The gas classification model supports the identification of gas-related conditions, while the occupancy and room occupancy models provide occupancy-related information for the monitored spaces. These model outputs allow the system to interpret sensor observations and provide information that would not be available from raw sensor values alone. The integration of real-time sensor data with historical information is an important aspect of the proposed system. MongoDB maintains the collected sensor records, allowing the system to access both current and previously recorded conditions. This supports the AI/ML processing and enables the dashboard to present changes in building conditions over time. The anomaly and risk assessment mechanisms further improve the interpretation of the monitored environment. Instead of requiring users to examine each sensor value independently, the system identifies conditions that may require attention and provides associated risk information and suggested actions. This supports a more proactive monitoring approach. The Digital Twin provides a unified representation of the monitored building spaces by associating the collected sensor information with the corresponding Reception, Manager Cabin, and Workspace. The integration of AI/ML outputs with the Digital Twin allows the virtual representation to provide not only the current physical state but also additional analytical information related to occupancy, gas conditions, and detected risks. The web-based dashboard provides a centralized interface through which users can observe the current building state, historical sensor information, model outputs, and recommendations. This integration improves the usability of the system by presenting multiple sources of information within a single monitoring environment. Overall, the developed system extends conventional IoT-based building monitoring by incorporating AI/ML-based prediction, anomaly analysis, and real-time recommendations. The combination of these components demonstrates the potential of integrating artificial intelligence with Digital Twin technology to support more informed and proactive building monitoring and management.

TABLE 7.4.1 Implemented Machine Learning Models

Model	Purpose	Output
Gas Classification Model	Classifies gas-related conditions	Gas condition/class
Occupancy Model	Predicts occupancy condition	Occupancy prediction and confidence
Room Occupancy Model	Estimates occupancy for monitored rooms	Occupancy count and confidence

TABLE 7.4.2 Model Input Parameters and Outputs

Model	Input Parameters	Output
Gas Classification Model	Gas sensor-related features and available environmental parameters	Gas classification
Occupancy Model	Environmental and occupancy-related sensor features	Occupancy prediction, confidence
Room Occupancy Model	Temperature, CO_2-related information, PIR state, and room information	Predicted occupancy count, confidence

TABLE 7.4.3 AI/ML Results Available from the System

Result Parameter	Description
Occupancy Prediction	Predicted occupancy condition generated by the ML model
Occupancy Count	Estimated number of occupants in a monitored room
Prediction Confidence	Confidence associated with the generated occupancy prediction
Model Name	Identifies the model used for generating the prediction
Risk Score	Indicates the assessed level of building/environmental risk
Detected Reasons	Conditions contributing to the identified risk
Suggested Action	Recommended response based on the detected condition

7.5 Data Visualizing and Reporting

The data visualization and reporting component provides a clear representation of the information collected and processed by the proposed AI-powered Digital Twin system. The React.js and Material UI-based dashboard presents real-time sensor readings, historical information, AI/ML results, anomaly information, and building-space conditions through an integrated user interface. The real-time sensor data collected from the Reception, Manager Cabin, and Workspace is displayed according to the corresponding building space. Parameters such as temperature, humidity, gas level, and motion status can be observed through the dashboard, allowing users to monitor the current condition of each space. Historical sensor data is also used for visualization and reporting. The stored records from MongoDB

can be represented using graphical and tabular formats to show changes in sensor parameters over time. This helps users identify variations and patterns that may not be apparent from individual real-time readings. The dashboard also presents the outputs generated by the implemented AI/ML models. Occupancy predictions, occupancy counts, prediction confidence, and gas-related classification results are integrated with the monitored sensor information. This provides users with both the measured physical condition and the corresponding analytical interpretation. Risk-related information is also incorporated into the reporting interface. The system can present the identified risk level, detected reasons, and suggested actions associated with the monitored condition. This allows users to understand not only the current sensor values but also the conditions that may require attention. The Digital Twin representation further improves data visualization by associating the sensor and analytical information with the corresponding physical building spaces. As new sensor readings are received, the virtual representation can be updated to reflect the latest available building condition. This provides a unified view of the physical environment and its digital counterpart. The visualization and reporting component therefore converts raw IoT data and AI/ML outputs into understandable information for users. The combination of real-time values, historical trends, Digital Twin representation, prediction results, and risk information provides a comprehensive view of building conditions and supports data-driven monitoring and decision-making.

CHAPTER 8

CONCLUSION AND FUTUREWORK

8.1 Conclusion

The proposed project, AI-Enabled Digital Twin for Intelligent Building Monitoring and Predictive Analytics Using IoT, successfully integrates IoT sensing, machine learning, anomaly detection, and Digital Twin technology into a unified building monitoring platform. The system establishes a connection between the physical building environment and its digital representation by continuously collecting sensor information from the monitored spaces. The implemented IoT layer uses ESP32-based sensing units with temperature, humidity, gas, and motion sensors to collect real-time information from the Reception, Manager Cabin, and Workspace. The collected data is transmitted through Wi-Fi to the FastAPI backend, validated, and stored in MongoDB. This provides a continuous and persistent data pipeline that supports both real-time monitoring and historical analysis. The AI/ML component extends the system beyond conventional sensor monitoring by applying multiple trained models for gas classification and occupancy-related prediction. The occupancy models provide prediction and confidence information, while the room occupancy model supports occupancy estimation at the individual building-space level. These outputs are combined with sensor information to provide a more meaningful interpretation of the monitored environment. The anomaly detection and risk assessment components further improve the intelligence of the system by identifying unusual conditions and providing associated risk information and suggested actions. This allows users to obtain actionable information instead of manually interpreting individual sensor readings. The Digital Twin provides a virtual representation of the monitored building spaces and synchronizes the digital environment with the latest available sensor information. The React.js and Material UI dashboard presents real-time sensor values, historical trends, AI/ML outputs, anomaly information, and recommendations through a centralized interface. Overall, the developed system demonstrates how IoT, AI/ML, and Digital Twin technologies can be integrated to provide intelligent building monitoring and predictive analytics. Compared with conventional monitoring approaches, the proposed system provides additional analytical capabilities that support proactive observation, condition assessment, and data-driven decision-making. The implementation establishes a foundation for further development of intelligent and scalable building-management applications.

8.2 Framework for Future Work

Although the proposed system provides an integrated platform for intelligent building monitoring, several enhancements can be considered for future development. The current implementation can be extended to support a larger number of sensors and building spaces, allowing the Digital Twin to represent larger and more complex building environments. Future work can include the integration of additional IoT sensors for parameters such as energy consumption, air quality, light intensity, noise level, and equipment operating conditions. Incorporating these parameters would provide a more comprehensive representation of building performance. The AI/ML component can also be enhanced by evaluating additional machine learning and deep learning techniques using larger and more diverse datasets. Continuous model retraining with newly collected building data can improve the adaptability of the predictive models to changing environmental and occupancy patterns.

Future versions can further develop predictive maintenance capabilities by analyzing equipment-related sensor data and identifying early indicators of possible failures. Energy-consumption prediction and optimization can also be incorporated to support energy-efficient building operation. The Digital Twin can be extended with a more detailed three-dimensional building representation and improved real-time synchronization. Integration with building automation and control systems could allow selected recommendations to be converted into automated actions, subject to appropriate safety and control mechanisms. The system can also be enhanced with advanced alerting, mobile accessibility, role-based dashboards, and more comprehensive reporting features. Cloud deployment and scalable infrastructure can further support monitoring across multiple buildings and locations. Therefore, future development can transform the current intelligent monitoring platform into a more comprehensive building-management solution capable of continuous prediction, optimization, automated decision support, and scalable Digital Twin-based operation.

APPENDICES

APPENDIX 1 (SOURCECODE)

```
/* ============================================================
1. ESP32 IoT SENSOR CODE
============================================================ */

#include <WiFi.h>
#include <HTTPClient.h>
#include <ArduinoJson.h>
#include <DHT.h>
#include <time.h>

#define DHT_TYPE DHT11

#define DHT_RECEPTION_PIN 4
#define DHT_WORKSPACE_PIN 5
#define DHT_MANAGER_PIN 18

#define PIR_RECEPTION_PIN 19
#define PIR_WORKSPACE_PIN 21
#define PIR_MANAGER_PIN 22

#define MQ2_PIN 34

#define BUZZER_PIN 23
#define WORKSPACE_LED_PIN 25

DHT dhtReception(
DHT_RECEPTION_PIN,
DHT_TYPE
);

DHT dhtWorkspace(
DHT_WORKSPACE_PIN,
DHT_TYPE
);

DHT dhtManager(
DHT_MANAGER_PIN,
DHT_TYPE
);

const char* WIFI_SSID = "YOUR_WIFI_SSID";
const char* WIFI_PASSWORD = "YOUR_WIFI_PASSWORD";

const char* SERVER_URL =
```

```
"http://YOUR_SERVER_IP:8000";

const char* SENSOR_ENDPOINT =
"/sensor-data/";

const long GMT_OFFSET_SEC = 19800;
const int DAYLIGHT_OFFSET_SEC = 0;

const int GAS_WARNING_THRESHOLD = 1500;
const int GAS_CRITICAL_THRESHOLD = 2000;

const unsigned long PIR_CHECK_INTERVAL = 100;
const unsigned long SENSOR_SEND_INTERVAL = 2000;
const unsigned long WIFI_RETRY_INTERVAL = 10000;

const unsigned long WORKSPACE_LED_ON_DELAY = 20;

const unsigned long BUZZER_ON_TIME = 5000;
const unsigned long BUZZER_OFF_TIME = 10000;

float receptionTemperature = 0;
float receptionHumidity = 0;

float workspaceTemperature = 0;
float workspaceHumidity = 0;

float managerTemperature = 0;
float managerHumidity = 0;

int gasLevel = 0;

bool receptionMotion = false;
bool workspaceMotion = false;
bool managerMotion = false;

bool previousWorkspaceMotion = false;
bool workspaceMotionEvent = false;

bool workspaceLedState = false;
unsigned long workspaceMotionStart = 0;

bool buzzerState = false;
unsigned long buzzerStart = 0;
bool buzzerCycle = false;

unsigned long lastSensorSend = 0;
unsigned long lastPIRCheck = 0;
unsigned long lastWiFiRetry = 0;
```

```
/* ------------------------------------------------------
WIFI
------------------------------------------------------ */

void connectWiFi()
{
Serial.println("Connecting to WiFi...");

WiFi.begin(
WIFI_SSID,
WIFI_PASSWORD
);

unsigned long startTime =
millis();

while (
WiFi.status() != WL_CONNECTED &&
millis() - startTime < 15000
)
{
delay(500);
Serial.print(".");
}

Serial.println();

if (
WiFi.status() ==
WL_CONNECTED
)
{
Serial.println(
"WiFi connected."
);

Serial.print(
"IP Address: "
);

Serial.println(
WiFi.localIP()
);
}
else
{
Serial.println(
"WiFi connection failed."
);
}
```

```
}


/* ------------------------------------------------------
TIME
------------------------------------------------------ */

void syncTime()
{
configTime(
GMT_OFFSET_SEC,
DAYLIGHT_OFFSET_SEC,
"pool.ntp.org",
"time.nist.gov"
);

Serial.println(
"Synchronizing time..."
);

struct tm timeInfo;

if (
getLocalTime(
&timeInfo
)
)
{
Serial.println(
"Time synchronized."
);
}
}


String getTimestamp()
{
struct tm timeInfo;

if (
!getLocalTime(
&timeInfo
)
)
{
return "";
}

char buffer[25];
```

```
strftime(
buffer,
sizeof(buffer),
"%Y-%m-%dT%H:%M:%S",
&timeInfo
);

return String(buffer);
}


/* ------------------------------------------------------
PIR
------------------------------------------------------ */

void readPIR()
{
receptionMotion =
digitalRead(
PIR_RECEPTION_PIN
);

workspaceMotion =
digitalRead(
PIR_WORKSPACE_PIN
);

managerMotion =
digitalRead(
PIR_MANAGER_PIN
);
}


void processWorkspacePIR()
{
if (
workspaceMotion &&
!previousWorkspaceMotion
)
{
workspaceMotionEvent =
true;

workspaceMotionStart =
millis();
}

if (
!workspaceMotion &&
```

```
previousWorkspaceMotion
)
{
workspaceMotionEvent =
false;

workspaceLedState =
false;

digitalWrite(
WORKSPACE_LED_PIN,
LOW
);
}

previousWorkspaceMotion =
workspaceMotion;
}


void updateWorkspaceLED()
{
if (
workspaceMotion &&
!workspaceLedState
)
{
if (
millis() -
workspaceMotionStart >=
WORKSPACE_LED_ON_DELAY
)
{
workspaceLedState =
true;

digitalWrite(
WORKSPACE_LED_PIN,
HIGH
);
}
}
}


/* ---------------------------------------------------
SENSOR READING
--------------------------------------------------- */

void readSensors()
```

```
{
float value;

value =
dhtReception.readTemperature();

if (
!isnan(value)
)
{
receptionTemperature =
value;
}

value =
dhtReception.readHumidity();

if (
!isnan(value)
)
{
receptionHumidity =
value;
}


value =
dhtWorkspace.readTemperature();

if (
!isnan(value)
)
{
workspaceTemperature =
value;
}

value =
dhtWorkspace.readHumidity();

if (
!isnan(value)
)
{
workspaceHumidity =
value;
}


value =
```

```
dhtManager.readTemperature();

if (
!isnan(value)
)
{
managerTemperature =
value;
}

value =
dhtManager.readHumidity();

if (
!isnan(value)
)
{
managerHumidity =
value;
}


gasLevel =
analogRead(
MQ2_PIN
);
}


/* ---------------------------------------------------
GAS WARNING
--------------------------------------------------- */

void processGasWarning()
{
if (
gasLevel >=
GAS_CRITICAL_THRESHOLD
)
{
Serial.println(
"Gas level: CRITICAL"
);
}
else if (
gasLevel >=
GAS_WARNING_THRESHOLD
)
{
Serial.println(
```

```
"Gas level: WARNING"
);
}
else
{
Serial.println(
"Gas level: NORMAL"
);
}
}


/* ------------------------------------------------------
BUZZER
------------------------------------------------------ */

void processBuzzer()
{
if (
gasLevel >=
GAS_CRITICAL_THRESHOLD
)
{
unsigned long elapsed =
millis() -
buzzerStart;

if (
!buzzerCycle
)
{
buzzerStart =
millis();

buzzerCycle =
true;

buzzerState =
true;

digitalWrite(
BUZZER_PIN,
HIGH
);

return;
}

if (
buzzerState &&
```

```
elapsed >=
BUZZER_ON_TIME
)
{
buzzerState =
false;

buzzerStart =
millis();

digitalWrite(
BUZZER_PIN,
LOW
);
}
else if (
!buzzerState &&
elapsed >=
BUZZER_OFF_TIME
)
{
buzzerState =
true;

buzzerStart =
millis();

digitalWrite(
BUZZER_PIN,
HIGH
);
}
}
else
{
buzzerCycle =
false;

buzzerState =
false;

digitalWrite(
BUZZER_PIN,
LOW
);
}
}
/* ============================================================
2. FASTAPI APPLICATION
============================================================ */
```

```
from contextlib import asynccontextmanager
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

from app.config.config import settings
from app.database.mongodb import (
connect_to_mongo,
close_mongo_connection
)
from app.utils.logger import logger
from app.api.router import api_router


@asynccontextmanager
async def lifespan(app: FastAPI):

logger.info(
"Starting AI-Powered Digital Twin API..."
)

await connect_to_mongo()

yield

logger.info(
"Shutting down API..."
)

await close_mongo_connection()


app = FastAPI(
title=settings.PROJECT_NAME,
version="1.0.0",
description="AI-Powered Digital Twin Backend API",
lifespan=lifespan
)


app.add_middleware(
CORSMiddleware,
allow_origins=["*"],
allow_credentials=True,
allow_methods=["*"],
allow_headers=["*"],
)


app.include_router(
```

```
api_router
)


@app.get("/")
async def root():

return {
"message":
"AI Digital Twin Backend Running",

"version":
"1.0.0",

"status":
"Healthy"
}


@app.get("/health")
async def health_check():

return {
"database":
"Connected",

"api":
"Running",

"status":
"Healthy"
}


@app.get("/info")
async def application_info():

return {
"project":
settings.PROJECT_NAME,

"version":
"1.0.0",

"environment":
"Development"
}


/* ============================================================
```

```
3. WEBSOCKET CONNECTION MANAGER
============================================================ */

from fastapi import WebSocket


class ConnectionManager:

def __init__(self):

self.active_connections = []


async def connect(
self,
websocket: WebSocket
):

await websocket.accept()

if (
websocket not in
self.active_connections
):
self.active_connections.append(
websocket
)


def disconnect(
self,
websocket: WebSocket
):

if (
websocket in
self.active_connections
):
self.active_connections.remove(
websocket
)
async def send_personal_message(
self,
message: dict,
websocket: WebSocket
):

try:

await websocket.send_json(
```

```
message
)
except Exception:

self.disconnect(
websocket
)

async def broadcast(
self,
message: dict
):

failed_connections = []

for connection in (
self.active_connections.copy()
):

try:

await connection.send_json(
message
)

except Exception:

failed_connections.append(
connection
)
for connection in failed_connections:

self.disconnect(
connection
)
manager = ConnectionManager()

/* ============================================================
6. REACT APPLICATION — App.jsx
============================================================ */

import {
BrowserRouter,
Routes,
Route
} from "react-router-dom";

import Dashboard from "./pages/Dashboard";
import Rooms from "./pages/Rooms";
import Prediction from "./pages/Prediction";
```

```
import Analytics from "./pages/Analytics";
import Alerts from "./pages/Alerts";
import Settings from "./pages/Settings";
import {
BuildingProvider
} from "./contexts/BuildingContext";


function App() {

return (
<BrowserRouter>
<BuildingProvider>
<Routes>

<Route
path="/"
element={
<Dashboard />
}
/>

<Route
path="/rooms"
element={
<Rooms />
}
/>

<Route
path="/prediction"
element={
<Prediction />
}
/>

<Route
path="/analytics"
element={
<Analytics />
}
/>
<Route
path="/alerts"
element={
<Alerts />
}
/>
<Route
path="/settings"
```

```
element={
<Settings />
}
/>
</Routes>
</BuildingProvider>
</BrowserRouter>
);
}
export default App;
```

APPENDIX 2 (SNAPSHOTS)



Fig. A2.1. Hardware Prototype

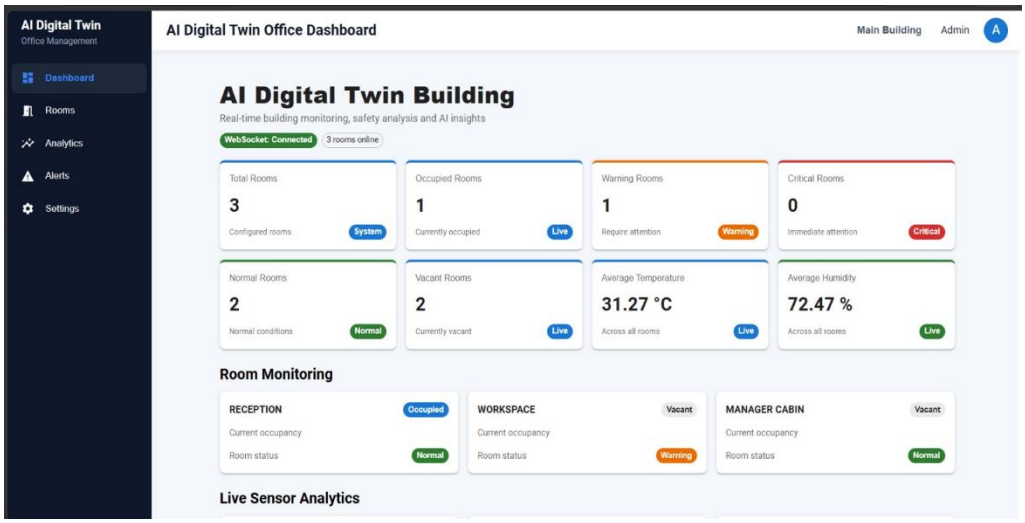


Fig. A2.2. Digital Twin dashboard for real-time building monitoring and visualization

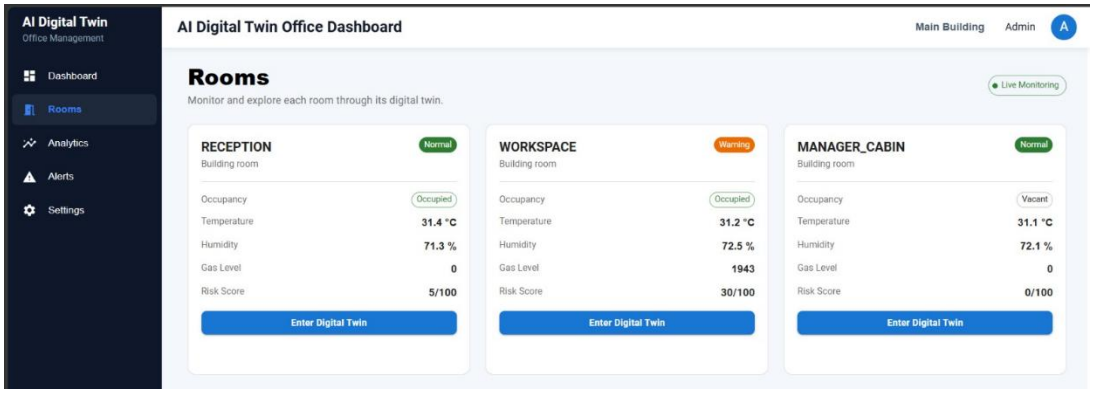


Fig. A2.3. Room-level Digital Twin dashboard showing environmental and occupancy conditions



Fig. A2.4. Historical sensor trends and analytical information displayed in the Digital Twin dashboard.

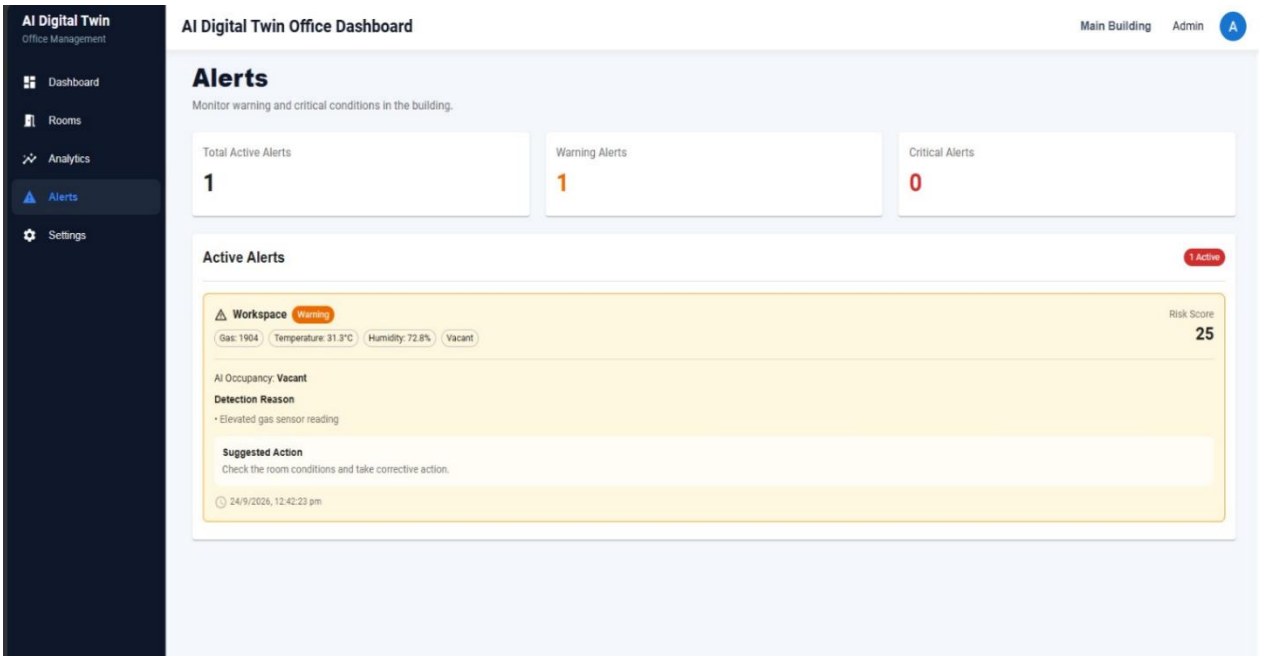


Fig. A2.5. Processed sensor conditions and alert states for the monitored rooms.

REFERENCES

1. Alanezi, K. and Mishra, S. (2023) 'Towards a Scalable Architecture for Building Digital Twins at the Edge', Proc. IEEE/ACM Symposium on Edge Computing (SEC), IEEE/ACM.

2. Alnaser, A.A., Maxi, M. and Elmousalami, H. (2024) 'AI-Powered Digital Twins and Internet of Things for Smart Cities and Sustainable Building Environment', Applied Sciences, Vol.14, No.24, Article No.12056.

3. Anwar, M.F., Waseem, F., Haque, M., Khan, M.A. and Abid, N. (2026) 'AI-Enabled Digital Twin Framework for Climate-Responsive and Sustainable Building Design', Proc. International Conference on Frontiers of Engineering and Emerging Technologies (FET), IEEE.

4. Boukaf, M., Fadli, F. and Meskin, N. (2024) 'A Comprehensive Review of Digital Twin Technology in Building Energy Consumption Forecasting', IEEE Access, Vol.12, pp.187778–187799.

5. Buonomano, A., Forzano, C., Giuzio, G.F., Palombo, A., Salzano, A. and Russo, G. (2024) 'Optimizing Building Performance: A Novel Digital Twin Framework for Enhanced Thermal Comfort', Proc. 3rd International Conference on Energy Transition in the Mediterranean Area (SyNERGY MED), IEEE.

6. Fang, B. (2024) 'Application of Artificial Intelligence and Big Data in Smart Buildings', Proc. International Conference on Electrical Drives, Power Electronics & Engineering (EDPEE), IEEE.

7. Furuta, S., Nakazato, J. and Tsukada, M. (2023) 'Web-Based BIM Platform for Building Digital Twin', Proc. 3rd IEEE International Conference on Digital Twins and Parallel Intelligence (DTPI), IEEE.

8. Gao, Q., Chu, Y., Peng, Z., Jin, Y., Ji, X., Ji, S., Ping, S., Xie, X., Zhu, X. and Yue, Y. (2024) 'Smart Building Management System Based on Digital Twin: A Case Study on Real-Time Environmental Monitoring and Thermal Comfort Prediction', Proc. IEEE International Symposium on Parallel and Distributed Processing with Applications (ISPA).

9. Ghaemi, A., Rezgui, Y., Petri, I., Beach, T. and Ghoroghi, A. (2025) 'AI and Digital Twin Applications in Building Energy Management: A State-of-the-Art Review', Proc. IEEE International Conference on Engineering, Technology and Innovation (ICE/ITMC).

10. Hidayat, W.N. et al. (2024) 'Digital Twin System for Smart Buildings Integrated with Blockchain and Mixed Reality Technology', Proc. IEEE International Symposium on Consumer Technology (ISCT), IEEE.

11. Islam, M.B., Guerrieri, A., Gravina, R. and Fortino, G. (2024) 'A Deep Learning Based Digital Twin for Indoor Temperature Prediction in Smart Buildings', Proc. IEEE Conference on Pervasive and Intelligent Computing (PICom).

12. Islam, M.B. et al. (2025) ‘iTempDT: AI-Powered Digital Twin for Forecasting Indoor Temperature in Smart Buildings’, Proc. IEEE Conference on Pervasive and Intelligent Computing (PICom), IEEE.

13. Motlagh, N.H. et al. (2024) ‘Digital Twins for Smart Spaces—Beyond IoT Analytics’, IEEE Internet of Things Journal, Vol.11, No.1, pp.573–583.

14. Mukhopadhyay, A., Talwar, N.R. and Biswas, P. (2024) ‘VR Digital Twin of Office Space with Computer Vision-Based Estimation of Room Occupancy and Power Consumption’, Discover Analytics, Vol.2, Article No.6.

15. Shahzad, A. et al. (2025) ‘Technologies and Techniques in Digital Twins for Real-Time Data Visualisation in Building Maintenance: A State-of-the-Art Review’, Journal of Infrastructure Intelligence and Resilience, Vol.4.

16. Swain, A., Mohapatra, S.K., Sahoo, S. and Dalei, A. (2024) ‘The Importance of a Digital Twin Strategy in the Context of Evaluating Occupants’ Interaction with Smart Building’, Proc. 2nd World Conference on Communication & Computing (WCONF), IEEE.

17. Tuutijärvi, J. et al. (2024) ‘AI-Assisted Decision Support for District Heating Demand Response’, Proc. IEEE 19th Conference on Industrial Electronics and Applications (ICIEA), IEEE.

18. Zaman, M., Saha, S., Addala, G.B., Zohrabi, N. and Abdelwahed, S. (2026) ‘An IoT-Driven Machine Learning Framework for Real-Time Monitoring in Smart Buildings’, Proc. IEEE International Conference on Smart Computing Workshops and Other Affiliated Events (SMARTCOMP Companion).

19. Zhang, X. (2026) ‘Design of Intelligent Construction Monitoring System for Building Engineering Based on Digital Twin, BIM, Computer Vision, and IoT Technologies’, Proc. 1st International Conference on AI, Data Science, Cyber Security and Smart Manufacturing for Sustainable Development (ICADCS), IEEE.

20. Zhou, X., Sun, K., Wang, J., Zhao, J., Feng, C., Yang, Y. and Zhou, W. (2023) ‘Computer Vision Enabled Building Digital Twin Using Building Information Model’, IEEE Transactions on Industrial Informatics, Vol.19, No.3, pp.2684–2692.

CONFERENCE DETAILS

Paper Name:

AI-Enabled Digital Twin for Intelligent Building Monitoring and Predictive Analytics Using IoT

Author Name(s):

1. DAKSHANAMURTHI S, 2. DHANUSH A, 3. HEMANTH KUMAR M P,

4. Mr. G. MuthuKumar, M.E., (Ph.D.)

Venue:

MEDICAPS UNIVERSITY (Hybrid)

Conference Title:

3rd International Conference on Computer, Communication and Control

Track:

Computer Science and Engineering

Paper ID:

960

Conference Date:

18th – 20th March 2027